



本科毕业设计（论文）

基于语义层的 Text-to-SQL 数据智能体系统设计与实现

学院（部、中心）：电信学部

专 业：自动化

班 级：自动化钱 2201（信息）

学生姓名：陈必纯

学 号：2222112595

指导教师：王平辉

2025 年 06 月

摘 要

自然语言交互的数据分析任务正在从“生成一条 SQL”转向“让智能体围绕数据资产检索、验证并维护可复用上下文”。在企业和科研场景中，关系数据库、表格文件、JSON 文档、说明文档、历史查询和业务规则往往共同构成分析上下文。大语言模型具备较强的语言理解与代码生成能力，但在真实数据环境中仍面临上下文组织困难、schema/value grounding（模式与值落地）不稳定、工具验证成本高以及知识难以跨问题复用等挑战。针对这些问题，本文设计并实现 Pontis：一种面向 Text-to-SQL 数据智能体的语义层，使智能体能够围绕数据资产逐步检索、验证和复用上下文，再完成自然语言查询与数据分析任务。

Pontis 以 Neo4j 属性图实现语义层，将文件、数据库、表、列、外键、统计摘要、语义描述、消歧规则和经验记录等统一建模为图实体及关系。系统通过自动化提取流程发布源事实，并生成列统计、值分布、列重叠、外键验证、语义摘要和向量索引等派生信息，同时通过探索智能体沉淀连接经验、字段消歧和局部任务提示。在此基础上，Pontis 设计了面向数据分析的查询智能体，将语义层检索、元数据读取和 SQL 查询等能力封装为工具，并通过轮次限制、实体检查、桥接表检查、消歧检查和值命中检查等护栏机制降低错误连接、未核实表列引用和无命中字面值风险。

Pontis 在 BIRD dev 全量 1534 道题上进行零样本评测，取得了 65.38% 的执行准确率，达到与若干专门 Text-to-SQL 基线相近的静态求解能力。本文的实验重点除提升执行准确率外，还分析了在准确率基础上不同系统的在线 token 结构、可缓存上下文、预处理摊销成本和长期复用特征。资源消耗方面，Pontis 将部分数据库理解转移至预处理阶段，并利用可缓存稳定上下文降低重复探索带来的非缓存输入。除主实验外，本文进一步讨论执行准确率作为单一指标的局限。错误归因显示，BIRD 错题中相当一部分样本并非单纯源于数据库探索不足，而是与数据集本身查询的自然语言模糊性、输出格式偏好、数据集隐含先验等因素有关。结果表明，Pontis 能够在保持优良静态准确率的同时，提供一种面向成本分析、上下文复用、错误归因和后续增量维护的数据智能体系统实现。

关 键 词：数据智能体；Text-to-SQL；大语言模型；语义层

ABSTRACT

Large language models have shown strong capabilities in natural language understanding and code generation, but they still struggle with real data environments because of implicit schema relationships, ambiguous domain terminology, unstable value grounding, and limited reuse of analysis experience. This thesis designs and implements Pontis, a semantic layer for Text-to-SQL data agents. Pontis introduces a queryable, updatable, and reusable layer between raw data and LLM agents, enabling agents to retrieve, verify, and reuse data context before answering natural language queries.

Pontis implements the semantic layer as a Neo4j property graph. Files, databases, tables, columns, foreign keys, statistical summaries, semantic descriptions, disambiguation hints, and reusable experience are modeled as graph entities and relationships. Deterministic extractors publish source facts and derived statistics, while exploration agents supplement connection hints, disambiguation information, and database-level README summaries. A tool-calling query agent then accesses the semantic layer through tools such as `find`, `meta`, and `query`. Guardrails including SQL entity checking, bridge-table checking, SQL disambiguation checking, value grounding, and final SQL rechecking are introduced to reduce incorrect joins, unsupported literal values, and unverified table or column references.

In a full zero-shot evaluation on the BIRD development set with 1,534 questions, no method uses gold SQL, question answers, or few-shot examples from the BIRD training split. Pontis achieves 65.38% execution accuracy, indicating that the semantic-layer system retains a competitive level of static Text-to-SQL solving ability. The main experimental emphasis of this thesis is not only execution accuracy, but also the cost structure of tool-calling Text-to-SQL systems. Most Pontis input tokens come from cacheable stable context; using normalized DeepSeek V4 Flash-style weights for cached input, uncached input, and output tokens, Pontis has a runtime price-converted cost of 16,132.8 per query and an amortized cost of 18,709.1 per query after preprocessing. Error attribution further shows that BIRD development errors involve mixed sources, including database exploration, output-shape preferences, hidden dataset priors, and underspecified natural-language semantics. These results suggest that execution accuracy alone is insufficient for evaluating long-running data agents, and that cost, reusable context, error attribution, and maintainability should be considered together.

KEY WORDS: large language model; agent; semantic layer; Text-to-SQL; data workspace

目 录

摘要	I
ABSTRACT	II
1 绪论	1
1.1 研究背景与意义	1
1.2 主要贡献	2
1.3 论文结构	3
2 相关工作	4
2.1 Text-to-SQL 基准与真实场景演进	4
2.2 Schema Linking 与元数据增强	5
2.3 智能体式 Text-to-SQL 与执行验证	6
2.4 语义层的图建模与接口抽象	6
2.5 评测协议与系统指标	7
3 问题定义与总体框架	8
3.1 Text-to-SQL 基本任务定义	8
3.2 传统 Text-to-SQL 范式的工程局限	8
3.3 核心挑战与设计目标	9
3.4 语义层总体框架	9
3.5 系统组成与数据流	11
3.6 语义层数据模型与统一接口	11
4 语义层构建方法	12
4.1 节点身份与关系建模	12
4.2 自动化脚本式信息提取	13
4.3 列统计与值分布	14
4.4 探索智能体	15
4.5 语义摘要与向量索引	16
4.6 增量刷新机制与成本控制	17
5 查询智能体与执行验证机制	19
5.1 智能体循环	19
5.2 工具集合	20
5.3 提示组装	20
5.4 护栏机制	21
5.5 运行示例	21

6 实验设计与结果分析	23
6.1 研究问题	23
6.2 数据集与评测指标	23
6.3 对比方法	24
6.4 主实验结果	25
6.5 预处理资源消耗	27
6.6 与基线方法的差异	27
6.7 数据库结构与内容变动下的增量更新对比	28
7 评测基准局限性分析	30
7.1 基本评测流程与执行准确率	30
7.2 SQL 多重可达性	31
7.3 Query 语义欠定性	32
7.4 Bash Agent 错误归因	33
7.5 标注错误与排行榜稳定性	34
7.6 从静态评测到交互式业务目标	34
7.7 独立同分布假设与 Query Log 评测	35
8 总结与展望	37
致谢	38
参考文献	39
附录 A Pontis 代码模块说明	43

1 绪论

1.1 研究背景与意义

数据正在成为组织运行、科研发现和软件系统决策的核心基础设施。与早期主要围绕单一关系数据库或固定报表展开的数据访问不同，现代数据工作流通常横跨数据仓库、湖仓、业务语义层、BI 仪表盘、数据管道、实验记录、代码仓库和团队知识库。数据消费者希望用自然语言提出探索性问题、追问分析过程、验证指标口径，并把得到的发现转化为后续决策或自动化动作。因此，数据系统面临的关键问题正在从“如何高效存储和查询数据”扩展为“如何让智能体理解、检索、验证并持续维护一个组织的数据知识”。

在这一背景下，数据智能体（data agent）成为学术界和工业界共同关注的方向。相比单次输入输出 Text-to-SQL 模型，数据智能体是能够围绕数据任务进行规划、调用工具、读取元数据、执行查询、验证结果并生成分析结论的系统。学术界的演进也体现了这一变化：Spider^[1] 和 BIRD^[2] 推动了 Text-to-SQL 从跨库泛化走向真实数据库和执行约束；Spider 2.0^[3] 进一步把评测对象扩展到真实数据应用和企业级工作流；InsightBench^[4] 与 Data Agent Benchmark^[5] 则将问题推进到多步洞察生成、异构数据源和企业数据问答。由此可见，自然语言数据分析正在从“单条 SQL 是否生成正确”转向“智能体能否在复杂数据环境中稳定完成分析任务”。

工业界也在快速把数据智能体嵌入主流数据平台。Microsoft Fabric data agent 被定位为 Fabric 中的对话式分析组件，能够连接 OneLake 中的 lakehouse、warehouse、semantic model 和 KQL 数据库，并在权限与治理约束下选择合适的数据源和查询工具^[6]。Snowflake Cortex Agents 通过 Cortex Analyst 和 Cortex Search 在结构化与非结构化数据之间进行工具路由，使应用可以在企业数据治理范围内完成 SQL 分析和文档检索^[7]。Databricks Genie Space 则要求领域专家用数据集、示例查询和文本指南配置面向业务团队的自然语言分析空间，并允许随着数据和用户问题变化持续更新语义知识^[8]。Tableau Next 和 agentic analytics 的提出也反映了 BI 平台从可视化报表向“从数据到洞察再到行动”的智能体化分析体验转变^[9]。

这些趋势表明，真实的数据智能体系统必须同时处理四类能力：第一，在复杂数据资产中定位相关实体和上下文；第二，理解组织内部的业务术语、指标口径和历史经验；第三，通过工具在真实数据上验证模型假设；第四，在安全、权限和治理边界内运行，并为后续问题积累可复用信息。由此可见，数据智能体的核心问题不仅包括 SQL 语法生成，还包括上下文组织、semantic grounding、工具编排、评测可靠性和长期信息维护。企业内部系统的实践研究也显示，有效的 Text-to-SQL 系统需要索引数据库元数据、历史查询、文档和代码等多源信息，基础表名和列名只构成其中一部分^[10-11]。

因此，将大语言模型直接用于真实数据环境仍面临以下困难。

第一，数据库模式和外部上下文过大。真实数据库可能包含大量表、列、视图和说明文档。若将完整 **schema**、样例值和说明文档全部纳入提示词，会造成上下文冗长、成本上升和注意力分散；若预先过滤 **schema**，又可能错误丢弃关键表列。尽管当前大模型上下文窗口不断扩大，数据分析任务在数据库规模较小时可以直接输入完整数据库 **schema**；但当企业 **schema** 继续扩张时，系统仍需要可靠的上下文控制机制来降低模型认知复杂度^[12]。

第二，关系、语义和值口径常常是隐式的。许多数据库没有完整外键约束，表间连接依赖命名约定、值域重叠、历史查询或说明文档。相同列名可能对应不同语义，不同列名也可能表达相同业务概念；即使表列选择正确，过滤值、枚举别名、日期边界和聚合口径也仍可能出错。相关研究表明，列统计、值分布、查询日志、业务规则和 LLM 生成的字段描述可以显著补足人工文档缺失的问题^[13-15]。

第三，模型必须在真实数据上验证假设。自然语言问题中的过滤值、枚举项、日期边界、比例分母和聚合粒度往往无法仅靠 **schema** 推断。SQL-PaLM、CHESS、ReFoRCE 和 APEX-SQL 等系统分别通过执行反馈、候选选择、多智能体协作和列探索来提升可靠性^[16-19]。这些工作说明，Text-to-SQL 正在从一次性生成转向带工具调用、检索、验证和修正的 **test-time framework**。

第四，系统评估需要同时关注效果和资源。BIRD 等基准提供了可复现的执行准确率比较，但智能体方法还会引入额外工具调用、候选生成、上下文检索和预处理成本。因此，系统评估不能只报告最终执行准确率，还需要记录 LLM 调用轮次、token 消耗、缓存命中和预处理开销等指标。

基于上述问题，真实场景中的自然语言数据分析需要同时考虑 SQL 生成、上下文组织、工具验证和可复用信息维护。Pontis 因此在原始数据和 LLM 智能体之间引入语义层，将 **schema**、统计、关系、摘要、消歧和经验统一表示为可查询上下文，使智能体能够通过工具逐步获取精确证据，减少每道题重新扫描完整数据库的成本。

1.2 主要贡献

围绕上述问题，本文设计并实现了 Pontis 语义层。主要贡献如下：

（1）提出一种面向 Text-to-SQL 数据智能体的语义层架构。该架构以 Neo4j 为持久化图存储，以 Cypher 为统一查询接口，以自动化脚本和探索智能体共同维护可复用的元数据、统计、摘要、关系和经验，避免在每次问答中直接拼接全部数据上下文。

（2）实现一套面向数据库语义层的信息提取与组织机制。系统能够自动暴露文件系统、SQLite 模式、表列结构和外键关系，并进一步生成列级统计、值样例、列重叠、AI 摘要和语义向量等派生信息。

（3）设计了结合工具调用和护栏机制的 LLM 数据智能体。系统将语义层检索、元数据查看、SQL 执行、文件搜索和图写入操作封装为工具，并通过 SQL 实体检查、桥接表检查和消歧检查减少常见数据库问答错误。

(4) 对 Pontis 与代表性基线方法进行统一实验比较和系统指标分析。实验将 Pontis 与 Bash Agent、CHESS-fast、Alpha-SQL、DeepEye-SQL 等方法放在同一评测协议下比较，在报告执行准确率的同时重点统计 token 消耗、LLM 调用轮次、缓存输入、非缓存输入和预处理摊销成本，并结合错误案例讨论静态执行准确率的解释边界。

1.3 论文结构

全文结构如下：第二章介绍相关工作；第三章定义 Text-to-SQL 数据智能体的基本任务和语义层总体框架；第四章阐述离线或预处理阶段的语义层构建方法及增量刷新机制；第五章介绍在线问答阶段的查询智能体与执行验证机制；第六章给出实验设计、主实验结果、资源成本、预处理摊销和增量更新对比；第七章讨论执行准确率作为单一评测指标的局限与数据智能体任务建模问题；第八章总结全文并展望未来工作。

2 相关工作

2.1 Text-to-SQL 基准与真实场景演进

Text-to-SQL 是自然语言处理与数据库系统交叉领域的重要任务，其目标是将自然语言问题转换为可执行 SQL。早期方法多依赖规则和语义解析，随后 Seq2SQL^[20] 等方法将神经网络和强化学习引入 SQL 生成。Spider^[1] 的提出推动了跨数据库泛化研究，RAT-SQL^[21] 通过关系感知编码器建模问题与数据库 schema 之间的关系，代表了在深度模型时代显式建模 schema linking（模式链接）的典型路线。

大语言模型出现后，Text-to-SQL 的研究重心从模型结构设计逐渐转向提示工程、检索增强、任务分解和执行反馈。C3^[22] 和 DAIL-SQL^[23] 系统比较了提示表示、示例选择和自修正等因素，DIN-SQL^[24] 则将复杂 SQL 生成拆分为 schema linking、查询分类和 SQL 生成等子任务。进一步地，MAC-SQL 将 Text-to-SQL 组织为多智能体协作过程，使用分解器、数据库选择和 SQL 修正等角色处理复杂问题^[25]。这些工作表明，强 LLM 不一定需要任务级微调才能完成复杂 SQL，但其效果高度依赖上下文组织和推理流程设计。BIRD^[2] 进一步引入更大的数据库、外部 evidence、真实值理解和 SQL 效率要求，使 Text-to-SQL 从语法映射转向数据库内容理解。

近年的基准进一步强调企业级复杂性和端到端数据分析能力：一类工作将任务扩展到真实数据应用、企业级 schema、查询日志和 SQL workflow，另一类工作则把评测对象从单条 SQL 扩展到多步业务洞察生成^[3-4,26]。这些工作共同说明，真实 Text-to-SQL 系统需要处理远大于教学数据集的 schema 空间、业务术语、私有规则、SQL 方言和多轮分析流程。Pontis 顺应这一趋势，面向 Text-to-SQL 数据智能体组织可检索、可验证和可维护的上下文。

与 Text-to-SQL 相邻的 data agent 研究进一步扩大了任务边界。DB-GPT 从 LLM 与数据库系统结合的角度讨论自动提示生成、数据库特定微调 and 数据库特定模型设计等问题^[27]；Data-Copilot 通过预先探索数据源并抽象可复用接口，减少实时请求中从零生成代码的错误^[28]；LIDA 将可视化生成建模为数据摘要、目标探索、可视化代码生成和信息图生成等多阶段过程^[29]；Data Interpreter 则面向数据科学任务，引入层次图规划、工具集成和反馈检查来完成端到端分析流程^[30]。此外，TAG 认为 Text-to-SQL 只能覆盖可直接表达为关系代数的一部分问题，真实数据问答还需要结合数据库执行、语言模型语义推理和最终答案生成^[31]。这些工作表明，本文关注的语义层不仅服务于单条 SQL 生成，也处在更广义的数据智能体和 AI+ 数据库交互研究脉络中。

2.2 Schema Linking 与元数据增强

Schema linking 负责将自然语言问题中的实体、属性和约束映射到数据库表列，是 Text-to-SQL 系统的核心环节。传统方法通常将 schema linking 作为 SQL 生成前的过滤步骤，但近期研究对这一环节提出了更细的分析。Maamari 等人的研究指出，当完整 schema 能放入上下文且模型推理能力足够强时，过早过滤 schema 可能因为召回错误而伤害生成结果^[12]。与此同时，面向大规模数据库的 schema linking 研究仍在继续强化上下文感知检索和双向匹配，以降低多表、多列环境下的召回遗漏风险^[32-33]。这一组结论说明，schema linking 的收益取决于 schema 规模、上下文窗口、模型能力和召回风险。对 Pontis 而言，系统设计重点从单纯裁剪 schema 扩展到可复用信息的组织，使智能体在需要时能够从语义层中逐步获取证据。

单纯依赖表名和列名也不足以解决真实场景中的语义问题。自动元数据抽取研究从数据库 profiling、列级统计、值样例、历史查询日志和 LLM 生成字段摘要中提取补充信息^[13]；schema refinement 工作则把原始数据库整理为更易解释的语义层，降低用户问题与底层 schema 之间的语义距离^[15]。面向多表问答的 schema graph 方法进一步说明，表列之间的连接关系、候选路径和人工确认信息适合用图结构表达，从而减少生成阶段对隐式推断的依赖^[14]。

长上下文能力为 Text-to-SQL 提供了另一条路线：系统可以把更多 schema、列样例、用户提示、SQL 文档或相似查询直接放入提示词。针对 NL2SQL 的长上下文研究表明，扩展上下文确实可以在部分场景中提升复杂 schema 理解能力，并带来准确率与延迟成本之间的新权衡^[34]。但一般长上下文研究也显示，模型对长输入中不同位置的信息利用存在稳定性问题，相关证据位于上下文中部时性能可能下降^[35]。Pontis 将可复用信息沉淀为语义层，并通过工具按需读取、验证和更新，以减少对一次性长提示词的依赖。

此外，schema linking 主要解决“选择哪些表列”的问题，过滤值、枚举映射、日期边界和聚合口径仍需要 value grounding 支持。BIRD 的设计已经强调数据库值和外部 evidence 的作用^[2]，企业系统中常见的业务简称、历史口径和隐式指标定义则进一步增加了 value grounding 难度。RubikSQL 将 NL2SQL 视为长期学习任务，持续维护业务术语、规则和失败经验^[36]；LinkedIn 的企业 Text-to-SQL 系统也把元数据、查询日志、文档和代码共同纳入知识图谱^[10]。这些工作表明，真实数据环境中的上下文由 schema、值、关系和经验共同构成。

Pontis 与这些工作的共同点在于重视元数据、值分布和关系信息；其设计特点在于将这些信息统一组织到语义层中，并通过工具接口让智能体按需读取，减少一次性拼接全部增强 schema 对上下文长度和信息位置的依赖。

2.3 智能体式 Text-to-SQL 与执行验证

工具调用是增强 LLM 能力的重要途径。在 Text-to-SQL 中，工具调用可以弥补模型静态知识不足的问题：模型可以执行 SQL 验证假设，通过搜索定位字段，通过样例值确认枚举，通过错误反馈修正语法和逻辑。

围绕 BIRD 和 Spider 2.0，已有方法逐渐形成多阶段、可验证的 test-time framework。SQL-PaLM^[16] 研究了 few-shot prompting、执行过滤、consistency decoding、指令微调 and 数据库内容使用对性能的影响；MAC-SQL^[25] 和 CHESS^[17] 使用多个 agent 或阶段分工处理 schema 定位、SQL 生成和修正；DeepEye-SQL 借鉴软件工程流程，将 value retrieval、schema linking、SQL generation、revision 和 selection 组织为可检查的流水线^[37]；ReFoRCE^[18] 引入表压缩、格式约束、自修正、共识选择和列探索；APEX-SQL^[19] 将生成过程组织为假设验证循环，通过真实数据探索落地语义。上述工作说明，复杂 Text-to-SQL 的性能同时取决于单次生成能力，以及检索、探索、验证和选择过程的组织方式。

除显式工具调用外，候选集选择和 test-time scaling 也是提升稳定性的常见路径。self-consistency^[38] 说明多条推理路径的一致性可以提高复杂推理可靠性；MCS-SQL 使用多提示和多选选择机制改善 Text-to-SQL 生成^[39]；CHASE-SQL 通过多路径生成和偏好优化选择器提高候选质量^[40]；Alpha-SQL 则将 SQL 构造过程建模为 MCTS 搜索，在零样本设置下迭代探索候选动作和部分 SQL 状态^[41]；Agentar-Scale-SQL 进一步从编排 test-time scaling 的角度讨论资源投入与准确率提升之间的关系^[42]。这类方法提升了复杂任务上的准确率，但通常也会增加 token 消耗和 LLM 调用轮次，因此需要和统一实验指标一起分析。

Pontis 同样采用工具调用范式，但其设计重点从多候选 SQL 构造转向可持久化的数据理解层。查询智能体的每次检索、元数据查看和 SQL 验证都围绕图实体展开，后续问题可以复用已有的统计、摘要、消歧和经验记录。

2.4 语义层的图建模与接口抽象

知识图谱和属性图是实现语义层的一类建模形式，适合表达数据库 schema、文件组织、表列归属、外键关系和业务概念之间的联系。传统知识图谱构建强调从文本中抽取实体和关系，而面向数据库语义层的图建模还需要处理结构化源事实和统计派生信息。企业 Text-to-SQL 实践已经显示，图结构可以把数据库元数据、历史查询、文档和代码等多源信息组织为可检索上下文，为 SQL 生成和错误修复提供依据^[10]。

在企业数据分析场景中，语义层的价值不仅在于存储概念关系，还在于为智能体提供可操作的数据地图。Schema graph 相关工作使用图结构减少模型对隐式连接关系的猜测^[14]；RubikSQL 将长期运行中沉淀的业务知识、错误修正和查询经验组织为可复用知识库^[36]；LinkedIn 的企业系统使用知识图谱统一数据资产、元数据和查询经验^[10]；

Uber QueryGPT 则结合向量数据库和相似查询检索，将自然语言问题和内部数据模型知识连接起来^[11]。这些实践说明，Text-to-SQL 系统需要一种位于数据库和模型之间的中间表示，用来承接不断变化的元数据、业务规则和历史经验。

Pontis 的语义层是围绕数据资产持续生成和更新的中间表示。文件系统和数据库 schema 可以作为源事实按需刷新，AI 摘要、列统计、列重叠和人工经验可以作为派生信息持久化。具体实现上，Pontis 使用知识图谱将当前数据库结构和已提取的分析经验统一到一个可查询图中。与只服务于单次 SQL 生成的增强 prompt 相比，语义层更适合跨问题复用、错误归因和后续更新。

2.5 评测协议与系统指标

Text-to-SQL 评测通常依赖执行准确率，即比较预测 SQL 与参考 SQL 的执行结果是否一致。相比 SQL 字符串比对，执行准确率能够减少语法表面差异带来的误判，因此已经成为 BIRD、Spider 等基准中的重要指标。针对 BIRD 的噪声分析指出，问题和 gold SQL 中的错误会影响模型表现和结论可靠性^[43]；进一步的 benchmark 审计研究也表明，标注错误可能显著改变模型排名^[44]。因此，执行准确率需要结合标注质量、任务语义和错误来源共同解释。

随着基准复杂度提高，评测对象也从“单条 SQL 是否匹配”扩展为“系统是否能稳定完成分析任务”。因此，除准确率外，系统比较还需要报告资源消耗，例如 LLM 调用轮次、输入输出 token、embedding token、预处理成本和运行时间。对于工具调用智能体和多阶段 pipeline，这些指标是理解系统可用性的重要组成部分。

Pontis 的评估同时记录执行准确率、llm_rounds、cached_input_tokens、uncached_input_tokens、output_tokens 和 total_tokens。这些指标能够区分可缓存的稳定内容和每题运行时新增内容，从而观察语义层是否减少重复输入，并提高 schema/value grounding 的可靠性。

3 问题定义与总体框架

真实数据分析任务正在从单次 SQL 生成转向带检索、工具调用和执行验证的数据智能体系统。本章在 Text-to-SQL 基本任务定义的基础上，分析传统生成范式在复杂数据环境中的工程局限，并给出 Pontis 的系统目标和总体框架。

本文所称语义层，特指 Pontis 在原始数据源与 LLM 之间维护的可查询、可更新中间表示；知识图谱或属性图表示该中间层采用的数据建模形式，Neo4j 图存储则表示具体的持久化后端。语义层中的信息按照来源和稳定性分为三类：源事实来自原始数据的确定性发布，包括目录、文件、数据库、表、列和外键等信息；派生信息由统计计算、摘要生成、embedding 或智能体探索产生；经验记录则保存人工或智能体确认后可跨问题复用的规则、消歧结论和错误归因。后文将在线回答用户问题的 agent 称为查询智能体，将通过 Pontis Agent、工具和 Workspace 访问系统并补充探索结果的 agent 称为探索智能体（explorer）。

3.1 Text-to-SQL 基本任务定义

Text-to-SQL 的基本问题形式为：给定数据库 $\mathcal{D}$ 和自然语言问题 q ，系统生成能够回答该问题的 SQL 查询 $\hat{s}$ 。这一输入输出关系可以统一写作：

$$\hat{s} = f(\mathcal{D}, q). \quad (3-1)$$

其中 f 表示由模型、提示词、检索策略、工具调用和解码过程共同组成的 Text-to-SQL 系统。该公式刻画静态基准中的基本输入输出关系；在具体系统内部，SQL 生成可以由一次性解码、多阶段流水线、搜索过程或工具调用智能体共同完成。不同方法之间的差异主要体现在系统如何组织上下文、是否进行预处理、如何调用工具以及如何利用执行反馈。

Pontis 在原始数据库和 LLM agent 之间构建语义层，将表列结构、统计信息、样例值、连接关系、库级 README、语义摘要、向量索引和消歧经验组织为可查询、可更新的信息。具体实现上，这些信息被写入 Neo4j 图存储并以图实体形式暴露给工具。在线求解时，查询智能体通过工具逐步读取相关内容并执行 SQL 验证，从而将数据库理解、上下文组织和执行检查显式纳入 Text-to-SQL 流程。

3.2 传统 Text-to-SQL 范式的工程局限

现有 Text-to-SQL 方法通常围绕单个自然语言问题组织推理过程：先为当前问题选择相关 schema、检索候选值或相似示例，再生成 SQL，并在部分方法中加入执行反馈、候选排序或自我修正。这类方法已经能够显著提升静态基准上的执行准确率，但

在面向长期使用的数据智能体系统时仍存在若干工程局限。

第一，上下文构造通常围绕当前 **query** 临时完成。真实数据环境中的有效上下文不只包括数据库 **schema**，还包括列统计、样例值、隐式连接关系、说明文档和历史经验。如果这些信息只在当前问题的 **prompt**、检索结果或候选 **SQL** 中短暂出现，系统很难把本次发现转化为后续问题可复用的资产。

第二，执行验证和错误修正往往服务于当前 **SQL**。自然语言问题中的过滤值、日期边界、聚合粒度和连接路径需要查询真实数据才能确认。即使系统在生成后使用执行结果修正 **SQL**，这些验证过程通常也只影响当前答案，较少沉淀为可查询的字段口径、连接经验或错误归因记录。

第三，预处理资产的维护粒度通常较粗。许多方法会构建 **schema** 索引、**value** 向量库、候选示例库或数据库摘要，但这些资产往往以文件、表或整库为单位维护，缺少稳定实体身份、来源记录和依赖传播机制。当数据库结构、值分布或业务说明发生变化时，系统难以判断哪些摘要、索引、关系和经验仍然有效。

因此，可查询、可维护的语义层主要解决长期数据智能体中的上下文组织、验证沉淀和局部更新问题；单句 **query** 的语义充分性和执行结果比对的解释边界，则属于评测协议本身的限制。

3.3 核心挑战与设计目标

基于上述问题，**Text-to-SQL** 数据智能体的核心挑战可以归纳为四类。

(1) 上下文定位。系统需要在复杂数据环境中找到与当前问题相关的数据库、表、列、文件和经验实体，同时避免把无关 **schema** 全部纳入提示词。

(2) **semantic grounding**。系统需要利用真实数据值、列统计、样例值和文档说明，把自然语言中的实体、枚举、时间范围和指标口径落到具体表列。

(3) 执行验证。系统需要通过 **SQL** 执行、结果检查和错误反馈验证假设，降低一次性生成 **SQL** 的偶然错误。

(4) 信息复用与更新。系统需要把可重复使用的统计、摘要、连接关系和消歧经验保存下来，并在数据库结构或内容变化时支持局部刷新。

Pontis 的设计目标是在原始数据和 **LLM agent** 之间提供一个可查询、可更新、可复用的语义层，并与现有专用 **Text-to-SQL pipeline** 形成互补。该语义层既要支持离线提取，也要支持在线问答；既要能被 **agent** 读取，也要能记录来源、身份和后续更新线索。

3.4 语义层总体框架

Pontis 的核心思想是在原始数据源和智能体之间建立语义层。系统整体由四个部分组成：原始数据源、语义层、信息提取器和查询智能体。

原始数据源包括 SQLite 数据库、表格文件、文本文件和说明文档。语义层负责保存数据资产的稳定身份、结构关系、统计信息、语义摘要、关系候选和向量索引。信息提取器在离线或预处理阶段生成这些内容；查询智能体则在在线阶段围绕用户问题调用语义层检索、元数据读取、SQL 查询和文本搜索等工具，逐步构造和验证最终 SQL。

从具体工作流程看，一个数据库进入 Pontis 后，系统首先发布数据库、表、列和外键等源事实；随后计算列统计、样例值、列重叠、语义摘要和向量；再由探索智能体补充难以完全规则化的 schema 摘要、关系线索、局部提示和库级 README 知识。在线问答时，查询智能体先根据用户问题检索相关实体，读取表列元数据和库级 README，再用 SQL 查询工具检查样例值或验证中间假设，最后输出 SQL。若最终 SQL 引用了未核实的实体、可能遗漏桥接表、包含数据库中无法命中的字面值，或触发局部消歧规则，护栏机制会阻断输出并要求查询智能体继续检查。

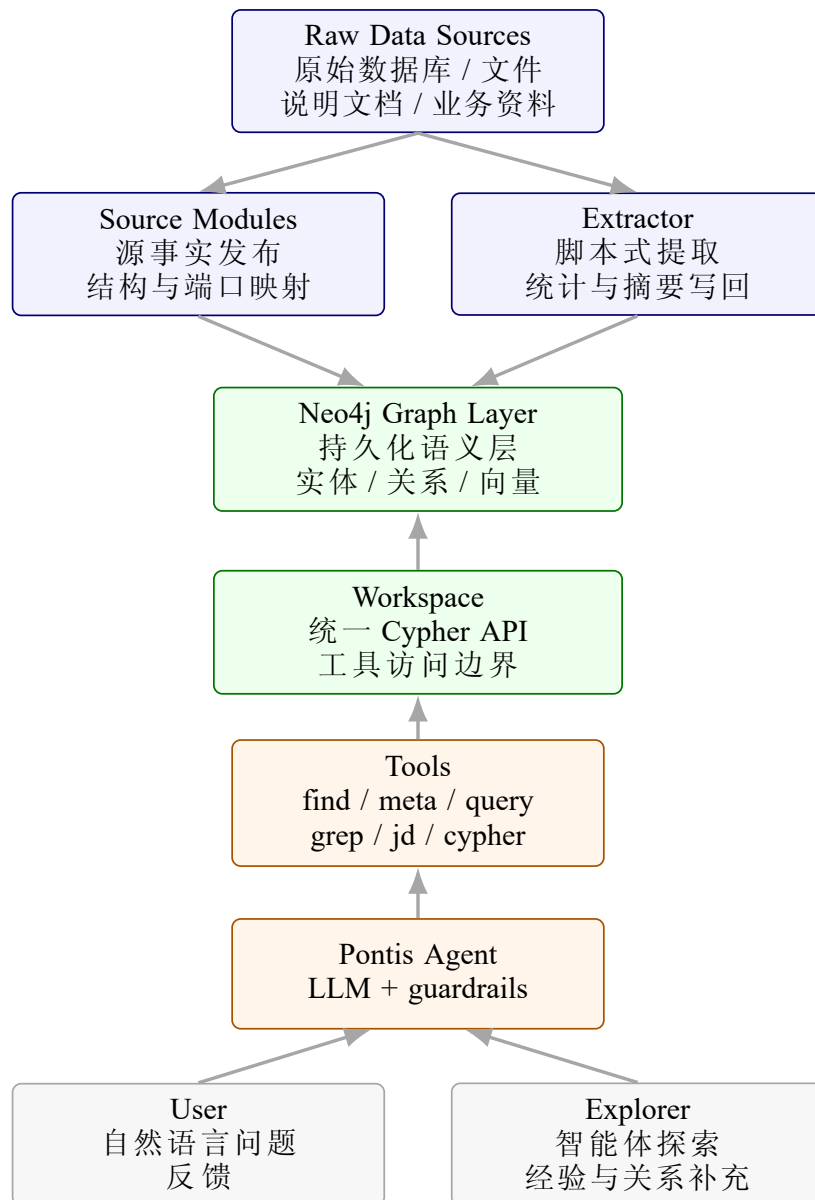


图 3-1 Pontis 工作空间与语义层架构

3.5 系统组成与数据流

Pontis 首先通过确定性的自动化脚本将数据源中的源事实发布为图节点和边，例如目录、文件、数据库、表、列和外键；随后继续在图存储之上生成统计、摘要、关系和向量等派生信息。Neo4j 负责持久化这些事实和派生信息，并提供 Cypher 查询接口。查询智能体通过工具访问语义层和原始数据，逐步完成自然语言分析任务。

系统的核心接口边界是 `Workspace.cypher(...)`。上层工具和提取流程均通过该接口访问图存储，避免直接依赖底层实现。这一接口将数据源刷新和图查询逻辑集中在工作空间层，从而降低工具层与存储层之间的耦合。

3.6 语义层数据模型与统一接口

Pontis 将数据资产表示为属性图。在典型的数据库 Text-to-SQL 场景中，主要涉及的实体类型包括：

- (1) 文件实体。表示工作空间目录中的文件，包含路径、文件类型和来源等属性。
- (2) 数据库实体。表示 SQLite 等数据库文件，作为表、视图和列的上级节点。
- (3) 表与视图实体。表示数据库表、视图或表格文件中的逻辑表，记录名称、来源和结构信息。
- (4) 列实体。表示表中的字段，记录列名、类型、样例值、统计信息和语义描述。
- (5) 外键和关系实体。表示显式外键、推断关系、列重叠和消歧信息。
- (6) 经验实体。表示人工或自动沉淀的规则、经验、错误分析结果和基准案例。

Pontis 使用 Neo4j 作为语义层的图存储后端。工作空间层对外暴露 Cypher 接口，源事实发布脚本和工具通过 Cypher 语句完成图实体写入、匹配和查询。选择 Neo4j 的主要原因包括：

- (1) Cypher 能直接表达图模式匹配、邻居遍历和多跳关系查询，适合表列关系、外键关系和经验实体检索。
- (2) 图存储与工具层解耦。工具只需要关心实体 ref、标签和查询结果，不需要自己维护图索引。
- (3) 跨数据环境路由更清晰。Pontis 可以为不同数据环境配置不同 Neo4j 端口或数据库，使同一套工具能够访问多个图谱实例。
- (4) 后续可利用 Neo4j 的索引、向量检索和查询优化能力，支撑更大规模数据环境。

4 语义层构建方法

离线或预处理阶段负责把原始数据资产转化为可查询、可复用、可增量维护的语义层。面向真实数据场景的自然语言分析任务，系统需要处理的上下文包括数据库 schema、列值分布、字段语义、隐式连接关系、业务术语、历史经验和非结构化说明文档等多类信息。传统 Text-to-SQL 方法通常在推理时临时拼接 schema 文本，而企业数据和真实基准中常见的弱文档、弱约束和多源数据使这种方式难以稳定覆盖必要证据。因此，Pontis 将这些信息组织进可增量更新的语义层，使查询智能体能够以实体和关系为单位读取上下文，减少每次问答时对完整数据库重新扫描的依赖。

现有研究将空值率、不同值数量、数据类型、值模式、函数依赖和包含依赖等信息视为理解数据集的重要元数据^[45]；Aurum 等数据发现系统通过图结构组织企业数据资产及其关系，以支持跨数据源发现和查询^[46]；自动元数据抽取和 schema graph 相关工作则进一步说明，列统计、字段描述、连接路径和人工确认信息能够补足原始 schema 的语义缺口^[13-14]。Pontis 将这些信息纳入同一语义层，为后续的检索、探索、验证和经验复用提供基础。

Pontis 的语义层构建采用分阶段、可增量的组织方式。首先，自动化脚本把文件系统、文本文件、表格文件和数据库 schema 转换为图节点与边，形成稳定的源事实视图；其次，提取流程在源事实之上补充列统计、值分布、结构模式、关系候选、语义摘要和向量索引；最后，探索智能体针对难以完全规则化的语义关系和消歧问题生成经验记录。三类结果共享同一图存储，但在来源、生成方式和可信度上保持区分。源事实和统计画像主要来自脚本，语义判断和连接经验主要来自探索智能体，从而降低错误推断对源事实的污染风险。

4.1 节点身份与关系建模

Pontis 在图存储中为主要数据资产维护稳定的实体身份。文件节点通常以路径合并，表和列节点使用数据库路径、表名和列名组成稳定引用，外键和关系实体则通过两端表列确定。所有节点都会获得存储层生成的内部 id，用于稳定边连接和工具显示。稳定身份是信息复用的前提：如果同一张表、同一列或同一段文本在不同运行中无法被识别为同一实体，统计结果、摘要、向量和智能体经验就难以增量更新，也无法可靠地被后续问题复用。

语义层的图建模遵循一个基本原则：具有独立语义或需要被检索、更新、引用的内容均建模为节点，例如表、列、外键、业务术语、历史经验和错误分析结果；边主要承担实体之间的连接与导航作用，复杂语义则保留在节点属性或关系实体节点中，便于 agent 直接探索和查询。

从数据发现角度看，图中的边主要作为被智能体利用的导航路径。文件到文本分片、数据库到表、表到列、列到统计、列到候选关系、实体到摘要和向量索引之间的

连接，使系统能够从一个自然语言查询逐步缩小到相关数据源、相关表列和可能的连接路径。**Schema graph** 工作表明，显式维护表列之间的图结构能够减少模型在多表问答中对隐式关系的即时猜测^[14]。**Pontis** 将这一思想扩展到数据库语义层，使关系型表、半结构化文件和智能体生成的经验记录可以处在同一个查询空间中。

4.2 自动化脚本式信息提取

自动化脚本式信息提取负责生成可以由数据源稳定推导或批量计算的信息。每个提取单元接收一个 **Workspace** 对象，通过 **Cypher** 和工具函数读取源事实，并将结果写回图存储。其任务既包括提取文件、文本、表格、数据库、表、列和外键等源事实，也包括在源事实之上生成统计、摘要、关系候选和向量索引。因此，这类脚本可以离线批量运行，也可以按实验需要选择性执行。

自动化脚本式信息提取将大量低层、重复、可计算的观察过程从查询智能体的在线循环中前移。对于一个新数据库，查询智能体如果在回答每个问题时都临时查看目录、浏览 **schema**、执行 **SELECT DISTINCT**、采样字段值并判断候选关系，会造成较高的 LLM 轮次和 **token** 消耗。自动化提取将这些步骤转化为可缓存的源事实和统计信息，使在线阶段更多地围绕已提取证据进行推理。数据画像和自动元数据抽取研究均表明，这类统计性和描述性元数据能够显著改善数据理解过程^[13,45]。

系统中的主要提取单元如表4-1所示。这里的“提取单元”既包括确定性脚本，也包括在预处理阶段运行的探索智能体；二者都通过同一工作空间接口写回语义层，但来源和可信度不同。

表 4-1 Pontis 主要信息提取单元

提取单元	功能
源事实发布模块	提取目录、文件、文本、SQLite 数据库、表、视图、列和外键等基础实体
表格与 JSON 结构模块	为 CSV/TSV/JSON 文件生成可检索结构、字段样例和模式摘要
数据库列统计模块	提取 SQLite 列统计、样例值、高频值、近似基数和空值比例
外键验证模块	验证数据库声明外键的有效性，并记录覆盖率和异常情况
列值重叠发现模块	计算列值重叠并生成潜在连接关系候选
列语义摘要模块	使用 LLM 根据列名、类型、统计和值样例生成数据库列摘要
schema_prepare	在预处理阶段为表列写入摘要，并确认高置信度关系线索
entity_hints	生成行粒度、字段落点、join 后果、格式风险等 SQL 决策提示
readme	将已生成的表列摘要、关系和风险压缩为库级 README
语义向量模块	为存在 detail 的实体生成 embedding ，并维护向量索引

自动化提取框架具有三个设计特征。第一，提取单元粒度较细，可以根据实验需要选择只运行源事实、统计、摘要、探索智能体或语义向量模块。第二，执行顺序可配置，既可以先运行基础结构提取和轻量统计，再运行关系发现与 LLM 摘要，也可以在基准实验中跳过高成本模块。第三，提取结果回写到同一个语义层中，后续工具和查询智能体无需感知提取过程的内部实现，只需查询图实体。

BIRD dev 实验采用受控预处理流水线，未启用 Pontis 中的全部 extractor。默认流程包括四组：静态模块 **db_column_stats_approx**、**db_fk_validate**、**db_column_overlap**；

LLM 列摘要模块 `ai_db_column_summary`；探索智能体模块 `agent_schema_prepare`、`agent_entity_hints`、`agent_readme`；以及 `semantic_embedding.agent_join_detect`、`agent_disambiguate`、文本分片和 CSV/JSON 摘要等属于通用或可选模块，不属于本次主实验默认路径。主实验结果对应上述默认路径，不覆盖这些可选模块。

在典型数据库中，自动化流水线可以分为四个阶段。第一阶段读取表、列和外键等基础节点，建立基础图；第二阶段扫描真实数据，生成列统计、样例值和高频值；第三阶段根据外键、值域重叠和统计特征生成关系候选；第四阶段使用 LLM 或 `embedding` 服务生成语义摘要和向量索引。各阶段的输出都以节点属性或派生实体的形式写回语义层，因此后续阶段可以复用前序阶段的结果。

4.3 列统计与值分布

列统计是 Text-to-SQL 中的重要上下文。仅靠列名往往无法判断一个字段真实含义，例如 `type`、`id`、`status` 等列名在不同表中可能含义不同；相反，字段中的真实取值、空值比例、基数和分布形态往往能揭示其业务角色。数据画像研究将这类信息归入单列元数据，并进一步讨论唯一列组合、函数依赖和包含依赖等跨列特征^[45]。Pontis 在系统实现上先聚焦对在线问答最直接有用的轻量统计，对数据库和表格文件中的列提取以下信息：

- （1）基本类型信息，包括 SQLite 声明类型和归一化类型。
- （2）空值、不同值数量、样例值和文本长度等轻量统计。
- （3）数值列的范围和分布摘要。
- （4）文本列的高频值，用于帮助模型理解枚举字段和业务取值。

这些统计信息会写入列实体的元数据，并在 `meta` 工具中呈现。与直接把整列数据交给 LLM 不同，统计摘要将大量原始值压缩成稳定、低成本上下文。对于大列，系统使用近似基数估计和流式高频项统计来降低预处理成本；对于文本列，系统保留少量去重样例和 `top-k` 高频值，使智能体能够判断枚举值、编码格式和自然语言问题中的值映射。

列统计同时服务于性能和准确率。性能上，它避免查询智能体为了理解一个枚举列而反复执行 `SELECT DISTINCT`；准确率上，它帮助模型判断过滤条件是否需要大小写转换、是否存在缩写、以及应使用编码列还是描述列。许多 `value grounding` 错误源于模型缺乏列中真实取值的证据，即使表列召回正确也可能发生。APEX-SQL 也强调通过真实数据探索验证列角色和查询假设^[19]。Pontis 将这类证据预先写入语义层，使查询智能体可以用一次元数据读取获得候选值、分布和异常线索，从而降低 SQL 探索轮次。

列统计还为后续语义摘要和关系发现提供输入。列摘要模块需要依赖样例值、高频值和基数判断字段是否为枚举、标识符、金额、日期或自然语言描述；列值重叠模块则需要利用 `distinct` 值和覆盖率判断两列是否可能构成连接路径。因此，统计画像构

成了 Pontis 语义层中从原始数据到高层解释信息的中间环节。

4.4 探索智能体

自动化脚本适合生成稳定事实和统计画像，但真实数据库场景中仍有一类信息难以完全规则化：字段语义需要结合上下文解释，隐式连接关系需要在全局 schema 中验证，同名列和近义字段需要根据样例值区分，长文档和说明材料中的业务口径也需要被压缩成可执行规则。Pontis 将承担这类任务的预处理智能体称为探索智能体。与最终回答用户问题的查询智能体相比，探索智能体面向离线或半离线阶段，其输出是写回语义层的可复用判断。

探索智能体采用专门化任务、受限工具和保守写入的组合，以避免通用 agent 在数据库中进行无约束探索。Toolformer 和 ReAct 说明了语言模型可以通过工具调用在外部环境中获取信息并逐步推理^[47-48]；CHESS、ReFoRCE 和 APEX-SQL 进一步把 Text-to-SQL 拆解为信息检索、schema 选择、列探索、候选生成和验证等子过程^[17-19]。Pontis 的探索智能体采用类似的任务分解思想，但输出对象由一次性 SQL 候选转为语义层中的长期信息。系统为探索智能体显式指定 prompt、工具集合和写入范围，通常只启用与任务相关的 find、meta、query、update_meta、create_entity 和 add_edge 等工具，并通过轮次护栏限制无界探索。

系统中的主要探索智能体如表4-2 所示。各类探索智能体按知识类型分工：摘要类探索智能体负责把表、列、文件和结构模式转化为自然语言语义；关系类探索智能体负责把候选连接线索提升为高置信度关系；提示类探索智能体负责记录会直接改变 SQL 选择的字段、值、粒度和 join 决策。

表 4-2 Pontis 主要探索智能体的设计职责

探索智能体	设计思路	写入内容
schema_prepare	面向数据库结构的预处理。先为表列补充摘要，再基于外键、overlap、样例值和表职责确认高置信度连接线索。	表、列、数据库、外键和关系实体的 brief/detail, 以及必要的 rel 实体。
entity_hints	面向 SQL 决策的提示抽取。重点维护行粒度、字段落点、事实表选择、join 后果、聚合粒度和格式风险。	基础实体的 hints 属性，以及邻接的 hint/disambig 实体。
readme	数据库信息压缩。复用已生成的表列摘要、关系、提示和数据质量线索，形成面向后续查询智能体的库级导航文档。	README 节点的数据库概览、主要对象、关系结构、数据质量风险和建议探索路径。
analyze/join_detect/disambiguate	通用可选探索智能体。分别承担全库理解、独立连接发现和独立消歧审计，适合在非 BIRD 或更重预处理场景中启用。	表列摘要、rel、disambig 和相关补充说明。
text_chunk/json_pattern/csv_summary	面向半结构化和非结构化文件的语义整理。	文本分片、JSON 模式摘要、CSV 字段摘要和文件级说明。

其中，关系信息体现了 Pontis 对图谱写入的保守态度。真实数据库中许多连接关

系没有显式外键，尤其在 **benchmark** 数据库、历史业务库和数据湖表格中，连接规则常常只存在于命名约定、值域重叠或业务经验中。数据发现系统通常需要在大量数据资产之间发现相关表、可连接列和可联合数据集^[46,49]；**schema graph** 方法也说明，多表问答需要显式维护可解释的连接路径，以降低生成阶段对 LLM 猜测的依赖^[14]。**Pontis** 因此同时利用三类信息发现关系：数据库声明外键、列值重叠候选和智能体验证后的 **rel** 实体。列值重叠模块批量生成候选证据，**schema_prepare** 或可选的 **join_detect** 再读取外键、**overlap**、列统计和样例值，检查是否已有确定关系覆盖、是否存在传递冗余、是否与全局结构冲突，只有在高置信度下才写入 **rel**。这种“候选生成 + 保守确认”的设计接近 **APEX-SQL** 中用真实数据探索验证假设的思路^[19]；**Pontis** 进一步将验证结果固化为可复用的关系记录。

entity_hints 和可选的 **disambiguate** 处理的是另一类高频错误：字段名相同并不必然表示语义相同，字段名不同也可能指向同一业务概念。例如多个表中都存在 **id**、**name** 或 **type** 时，最终 SQL 生成前需要确认字段来源；而 **Free Meal Count** 与 **FRPM Count** 这类近义字段也可能存在包含关系或统计口径差异。相关探索智能体会按列名、近名模式和 SQL 决策风险收集候选实体，再通过列统计、样例值、表职责和已有关系判断是否确有歧义。确认后，系统创建 **disambig/hint** 实体，并把短结论写入基础实体 **hints**，后续 SQL 消歧护栏会在高风险输出时提醒查询智能体读取这些信息。**Agentic schema refinement** 强调通过语义层整理降低用户问题与底层 **schema** 的距离^[15]；**Pontis** 的提示和消歧探索智能体可以看作面向局部歧义的轻量 **schema refinement**。

schema_prepare、**readme** 和可选的 **analyze/text_chunk** 则负责把数据资产转化为可检索、可复用的语义描述。**schema_prepare** 按数据库入口、核心表、依赖表、列统计和关系实体的顺序建立理解，并优先读取精确属性，以避免一次性展开完整数据上下文导致 **prompt** 膨胀；文本探索智能体则为长文档创建语义分片。这些设计对应于数据画像、语义类型检测和数据湖发现研究中的共同观点：数据理解需要结合结构、统计、值模式和上下文表示，而不能仅依赖文件名或字段名^[45,49-50]。

readme 体现了 **Pontis** 对长期信息积累的利用。其把已有摘要、关系、消歧和数据质量线索压缩成库级 **README**，使后续查询智能体进入数据库时可以直接读取已沉淀的导航信息。**RubikSQL** 将工业 **NL2SQL** 视为长期学习任务，强调通过数据库画像、规则挖掘和知识库维护持续积累领域知识^[36]；**LinkedIn** 和 **Uber** 的企业实践也显示，文档、业务规则和知识图谱能够改善自然语言到 SQL 的稳定性^[10-11]。

4.5 语义摘要与向量索引

Pontis 中的 **brief** 和 **detail** 是面向智能体的核心文本属性。列统计提供字段取值层面的证据，AI 摘要进一步补充字段或表的业务语义。系统会对表、列和文本文件生成摘要，并通过语义向量模块为 **detail** 文本生成向量。语义类型检测和表理解研究表明，字段含义通常需要结合列名、值分布、字符模式和表上下文判断，单独依赖表头

字符串容易产生误判^[50]。因此，Pontis 的摘要生成会把统计画像、样例值、上下游关系和文件上下文压缩成适合智能体读取的自然语言说明。

列摘要模块在生成列说明时输入列名、类型、基数、空值比例、数值范围、样例值和高频值，并要求输出稳定的业务语义，避免生成易过时的具体统计数字。对于表摘要，系统会聚合同表列信息和外键关系，描述表的职责、粒度和常见关联路径。对于文本文件，系统会提取文本分片摘要，使非结构化说明材料也能进入同一个检索空间。

语义向量模块只对存在非空 `detail` 的节点生成 `embedding`，并记录模型名、向量维度和 `detail` 文本 `hash`。当 `detail` 未变化且 `embedding` 配置一致时，模块会跳过该节点，避免重复请求 `embedding` 服务。向量写回 Neo4j 后，系统按节点标签创建向量索引，使 `find(ref, query)` 从名称匹配扩展为基于自然语言语义的实体检索。例如，当问题涉及“percentage”“ratio”“foreign player”或“school SAT score”时，系统可以召回对应表列、规则说明或历史案例。Starmie 等数据发现工作利用列表示和高效向量索引提升数据湖中的语义检索效率^[49]；Pontis 在规模较小的语义层图存储中采用相同方向的思想，使智能体更快聚焦到候选实体，减少全量浏览 `schema` 的成本。

4.6 增量刷新机制与成本控制

语义层构建的成本主要来自三部分：源扫描、数据统计和 LLM/embedding 调用。Pontis 对这三部分分别采用不同策略。源扫描通过数据源 `fingerprint`（指纹）和刷新缓存避免重复发布未变化的数据源；数据统计尽量使用单次扫描、近似基数和 `top-k` 结构压缩原始值；LLM 摘要和语义向量则通过缺失字段检查、文本 `hash` 和批量请求减少重复调用。

数据库结构或内容变化时，Pontis 将维护动作落实到语义层的节点、边和派生信息上。数据库、表、列、统计、摘要、关系、库级 `README`、消歧规则和 `embedding` 均可携带来源、时间戳、`fingerprint` 或文本 `hash`。当某张表新增列时，系统刷新该表及其相邻列的结构节点、统计、摘要和相关 `embedding`；当某个枚举列的值分布变化时，系统将该列统计、相关 `value grounding` 经验和依赖的库级 `README` 片段标记为 `stale`（过期）；当外键或隐式连接发生变化时，系统沿关系节点传播脏标记，并重新运行局部 `join` 探索。这样，数据库变动可以被转化为局部图更新，而不是每次都重新构建整库上下文。

具体而言，Pontis 的增量刷新由六类机制组成：第一，源数据变更检测，通过 `schema fingerprint`、行数和列分布摘要识别结构与内容变化；第二，派生节点的 `provenance`（来源记录），使每条摘要、关系或经验记录能够追溯依赖的源节点；第三，`dirty flag`（脏标记）与依赖传播，用于判断哪些派生信息已经过期；第四，局部 `extractor` 调度，使系统只重跑受影响的统计、摘要、向量和探索智能体；第五，`stale-safe`（过期保护）查询模式，在在线回答时向查询智能体暴露过期状态，避免模型把失效信息当作确定事实；第六，人工确认记录锁定，用于保留高置信度业务规则，避免数据库短期噪声覆

盖已经确认的语义经验。上述机制使 Pontis 能够在数据库持续变化时维护语义层的一致性，并把维护成本限制在受影响的表、列和关系子图内。

这种设计使 Pontis 能在预处理成本和在线查询成本之间进行明确取舍。预处理阶段预先投入计算成本生成统计、摘要和关系线索，可以减少查询智能体在线阶段的工具调用轮次、SQL 探索次数和输入 token。不同数据库可以选择不同流水线深度：在轻量场景中可以只运行源事实发布和列统计；在复杂 Text-to-SQL 基准实验中，则可以追加列重叠、AI 摘要、向量索引和探索智能体，以换取更强的 schema linking、value grounding 和 join-path 可靠性。

5 查询智能体与执行验证机制

在线问答阶段，查询智能体需要围绕具体自然语言问题组装上下文、调用工具、执行 SQL 验证，并在护栏反馈下修正最终输出。与离线语义层构建不同，这一阶段关注运行时的信息读取、假设验证和输出约束。

5.1 智能体循环

Pontis 查询智能体采用工具调用循环。每轮循环中，系统将用户问题、系统提示、工具定义和历史消息发送给 LLM；若 LLM 返回工具调用，Pontis 执行对应工具并把结果作为下一轮输入；若 LLM 返回最终文本，则暂时结束循环等待用户反馈。

记用户问题为 q ，数据库为 $\mathcal{D}$ ，当前语义层为 $\mathcal{K}$ ，第 t 轮前的对话、工具结果和护栏反馈状态为 h_{t-1} 。Pontis 的在线循环可以表示为：

$$a_t = \pi(\mathcal{D}, \mathcal{K}, h_{t-1}), \quad h_t = \Phi(h_{t-1}, a_t, o_t). \quad (5-1)$$

其中 π 表示由 LLM、提示词、工具定义和解码策略共同形成的动作生成策略， a_t 表示第 t 轮动作，可以是工具调用、最终 SQL 或自然语言回答； o_t 表示该动作产生的观测结果，例如工具返回、SQL 执行结果、护栏阻断反馈或用户反馈； Φ 表示状态更新函数。若 a_t 是工具调用，则系统执行工具得到 o_t 并更新 h_t ；若 $a_t = \text{Final}(\hat{s})$ ，则输出最终 SQL $\hat{s}$ 并结束本轮在线求解。在 BIRD 主实验中， $\mathcal{K}$ 在单题求解过程中保持只读；在更一般的数据智能体场景中，写入工具可以把确认过的经验进一步沉淀回语义层。

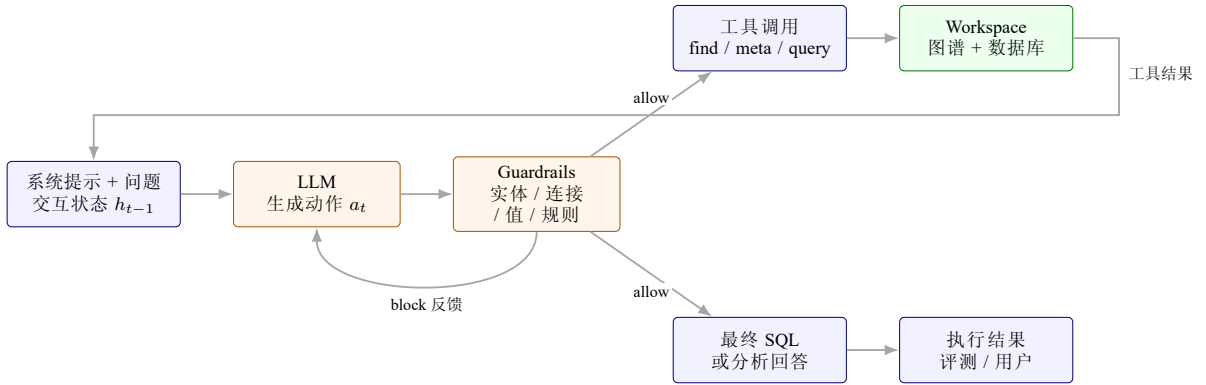


图 5-1 查询智能体与护栏执行流程

查询智能体的在线成本主要由 cached input token、uncached input token、output token 和 LLM 调用轮次共同决定。cached input token 对应可被 token 缓存命中的稳定前缀，uncached input token 对应每题运行时新增上下文，output token 对应模型生成内容，LLM 调用轮次则反映串行模型交互次数。

5.2 工具集合

Pontis 将工具划分为只读工具、写入工具和子智能体工具。通用只读配置包含实体查找、元数据读取、文件读取、JSON 下钻、内容搜索、SQL 查询、图查询和受限命令执行等能力。写入模式增加经验实体创建、元数据更新、边添加和删除能力，用于经验沉淀和语义层维护。BIRD 主实验将在线求解工具限制为 `find`、`meta` 和 `query`，以减少任意文件读取、`shell` 探索和额外上下文对实验结果的影响。

表 5-1 Pontis 主要工具

工具	类别	作用
<code>find</code>	语义层检索	按 <code>ref</code> 模式和语义 <code>query</code> 查找实体
<code>meta</code>	元数据	查看实体属性和邻居关系
<code>query</code>	数据查询	在数据库、CSV/TSV、JSON 或工作空间范围内执行只读 SQL
<code>grep</code>	文本搜索	在文件实体范围内搜索内容
<code>read</code>	文件读取	读取文本文件指定行范围
<code>jd</code>	JSON 下钻	查看 JSON 文件结构和指定路径内容
<code>bash</code>	命令执行	执行受限 <code>shell</code> 命令
<code>cypher</code>	图查询	直接执行只读图查询
<code>create_entity</code>	图写入	创建经验实体
<code>update_meta</code>	图写入	更新实体元数据
<code>add_edge</code>	图写入	添加实体关系边
<code>delete</code>	图写入	删除实体
<code>agent</code>	子任务	调用子智能体完成局部任务

其中，`find`、`meta` 和 `query` 是 Text-to-SQL 场景中最核心的三个工具。`find` 用于缩小候选表列或知识范围，`meta` 用于精读实体语义和邻居关系，`query` 用于验证 SQL 假设和检查样例值。当前 `query` 工具不仅可以针对单个数据库文件执行只读 SQL，也可以把 CSV/TSV 或可表格化 JSON 映射为临时 SQLite 表；当 `ref` 为工作空间时，工具会注册其中的数据库、表格和 JSON 表，并给出可用表名提示，从而支持更一般的数据问答。

5.3 提示组装

Pontis 的提示词由稳定系统提示和任务实例提示共同构成。系统提示保留跨任务复用且对智能体理解数据库上下文必要的内容，包括基础身份、工具说明、语义层本体、SQL 规范、`effort` 约束、护栏规则、数据库上下文和库级 README 摘要。

这种分层组装方式具有两个作用。第一，稳定系统提示可以在同一类任务中复用，减少重复 `token` 并有利于上下文缓存；第二，任务实例提示保留具体场景的约束，避免将评测输出格式、预处理写图策略或错误归因规则混入所有智能体实例。工具相关提示词由创建智能体时显式传入的工具集合控制。

源码中的真实组装顺序为 `base`、`tool`、`ontology`、`sql`、`effort`、`guardrail`、`project`、`readme`。其中前置段落位于同一工具和护栏配置下高度稳定，被放在前缀以提高缓存命中；数据库上下文和库级 README 更容易随数据库变化，因此被放在后部。

BIRD benchmark 额外传入独立的任务提示、question、evidence 和输出格式约束，作为每题运行时上下文进入对话历史。

5.4 护栏机制

Pontis 的护栏层位于 LLM 输出和工具执行之间。每个护栏独立检查待执行工具调用或最终文本输出，并给出允许、警告或阻断裁决。多个护栏的裁决按阻断高于警告、警告高于允许的优先级聚合。

表 5-2 Pontis 护栏机制

护栏	文件	作用
RoundLimit	round_limit.py	达到轮次上限后阻止继续调用工具
ExplorationCheck	exploration_check.py	阻止过宽泛的全图探索
ToolAbuse	tool_abuse.py	限制指定工具的过量使用
SQLEntityCheck	sql_check.py	检查 SQL 引用表列是否已读且存在
BridgeTableCheck	sql_join_check.py	检查多表连接是否遗漏桥接关系
SQLDisambigCheck	sql_disambig_check.py	提醒同名列和局部术语消歧
SQLValueGroundingCheck	sql_value_grounding_check.py	检查最终 SQL 中字面过滤值是否能在真实列值中命中
BirdReadmeFinalRecheck	bird_readme_final_recheck.py	在 BIRD 最终 SQL 输出前回灌规则说明并强制自检一次
MetaDisambigPrefetch	meta_disambig_prefetch.py	读取 meta 后异步预取邻接消歧和 hint 上下文

护栏在高风险节点向模型提供结构化反馈。例如，模型若尝试输出最终 SQL，但其中引用的表列从未通过 meta 阅读，SQL 实体检查会阻断最终输出并要求模型先核实实体；若 join 中涉及可能需要桥接表的列对，桥接表检查会提示相关关系实体；若 SQL 中出现与消歧实体相关的术语，消歧检查会要求模型读取对应解释；若最终 SQL 中出现 = 或 LIKE 字面值过滤且在目标列中命中 0 行，value grounding 检查会把高频值作为反馈返回给模型。

不同运行模式启用的护栏集合存在差异。通用只读配置默认使用轮次限制、探索检查、工具滥用限制、SQL 实体检查、桥接表检查和 SQL 消歧检查。BIRD 基准求解阶段在此基础上显式加入 SQLValueGroundingCheck 和 BirdReadmeFinalRecheck。后者的实现方式是：当模型准备输出最终 SQL 后，系统将 SQL 写法约束作为 release checklist 回灌一次；无论模型先前 SQL 是否看似可行，系统都会阻断一次最终输出，要求模型重新检查输出列、排序、聚合、DISTINCT、LIMIT 和 SQLite 写法。该机制引入一次额外 LLM 调用，并将最终输出检查集中在 SQL 生成末端。

5.5 运行示例

以 student_club 数据库中“closed events 的 spend-to-budget ratio”类问题为例，Pontis 的在线求解过程可以分为检索、精读、验证、护栏反馈和最终输出五个阶段。表5-3 给出了该过程的简化轨迹。

表 5-3 Pontis 在线运行示例

阶段	系统行为
find	查询智能体用“event”, “budget”, “spend”, “closed”等关键词检索语义层，定位 event、budget 以及二者之间的候选关系。
meta	智能体读取候选表列的 brief/detail、列统计、样例值和邻接关系，确认 budget.spent、budget.amount、budget.event_status 和 budget.link_to_event 的含义。
query	智能体执行只读 SQL 检查 event_status='Closed' 的命中情况，并验证 link_to_event 能够连接到 event.event_id。
护栏反馈	若候选 SQL 引用了未通过 meta 核实的列，SQLEntityCheck 阻断最终输出；若字面值 Closed 在目标列中无法命中，SQLValueGroundingCheck 返回高频值并要求修正。
最终 SQL	通过实体、连接和值检查后，智能体输出只包含目标列和必要排序的 SQL，例如围绕 budget 与 event 连接并计算 spent / amount 的查询。

该示例体现了 Pontis 的实现路径：语义层负责缩小候选对象和提供元数据证据，query 工具负责验证真实数据状态，护栏负责在最终 SQL 前检查实体、连接和字面值风险。最终输出不是一次性从完整 schema 中生成，而是在工具观测和护栏反馈约束下逐步形成。

6 实验设计与结果分析

6.1 研究问题

实验与分析围绕以下问题展开：

(1) Pontis 在 BIRD dev 静态评测中是否能够达到可接受的执行准确率，从而说明语义层式系统具备基本 Text-to-SQL 求解能力？

(2) Pontis 在解题过程中产生的 LLM 调用轮次、缓存输入、非缓存输入和输出 token 具有什么结构特征？

(3) 语义层预处理如何改变 Text-to-SQL 系统的离线成本、在线成本和可复用上下文分布？

(4) 在同一数据库被长期重复查询或数据库结构与内容发生变化时，Pontis 与基线方法的更新粒度和维护成本有什么差异？

6.2 数据集与评测指标

实验采用 BIRD 开发集作为主要数据。BIRD dev 包含 11 个数据库和 1534 道自然语言问题，提供 evidence、标准 SQL 和难度标签，适合评估真实数据库问答能力。本文报告执行准确率，因为它是当前 Text-to-SQL 领域最常用且最可复现的横向比较指标；但本文不将执行准确率最大化作为唯一目标。BIRD 中部分题目具有明显的数据集标注特征，百分比、比例、输出列、排序和 COUNT DISTINCT 等口径可能存在多种业务上合理的解释。因此，准确度在本文中主要用于判断系统是否具备基本求解能力，实验分析的重点还包括 token 资源消耗、成本结构、上下文复用和错误案例。

主准确度指标为执行准确率（Execution Accuracy, EX），即预测 SQL 与 gold SQL 在同一数据库上执行后的结果集合是否一致。实验记录以下指标：

表 6-1 实验指标

指标	含义
Accuracy	SQL 查询结果比对得到的执行准确率
cached_input_tokens/Q	每题可被 token cache 命中的稳定输入上下文
uncached_input_tokens/Q	每题运行时新增、难以跨题缓存的输入上下文
output_tokens/Q	每题模型输出 token，包括工具调用参数、解释文本和最终 SQL
LLM Rounds/Q	每题串行 LLM 调用轮次
Total Tokens/Q	每题总 token 消耗，包含 cached input、uncached input 和 output token

不同方法的成本需要结合 token 结构分析。对于工具调用智能体而言，每一轮 LLM 调用都会重新发送一部分稳定上下文和一部分动态上下文。稳定上下文通常包括系统提示、工具说明、库级 README、数据库级提示和输出格式约束；动态上下文则包括当前问题、检索结果、工具返回、SQL 执行结果和前几轮模型输出。两者的复用

性质和实际成本不同，如果全部合并为 input tokens，就无法判断一个方法的成本来自稳定提示词过长，还是来自运行时探索过多。

实验将输入 token 拆分为 cached input 和 uncached input 两类。前者表示可被 token cache 命中的稳定输入前缀，后者表示每题运行时新增、难以跨题复用的输入内容。Output tokens 单独记录模型生成的工具调用、分析文本和最终 SQL。Total tokens 作为总量参考，成本分析主要考察缓存输入、非缓存输入和输出三类结构。单题 token 消耗可近似分解为：

$$C_{\text{token}} = C_{\text{cached}} + C_{\text{uncached}} + C_{\text{output}}. \quad (6-1)$$

其中 C_{cached} 表示系统提示、工具定义、库级 README 等可被 token 缓存命中的稳定前缀； C_{uncached} 表示每题运行时产生的问题、检索结果、工具返回和历史消息； C_{output} 表示模型生成的工具调用、分析文本和最终 SQL。不同 API 提供商对缓存输入、非缓存输入和输出 token 的计费方式不同，价格折算采用如下参数形式：

$$C_{\text{price}} = p_{\text{cached}}C_{\text{cached}} + p_{\text{uncached}}C_{\text{uncached}} + p_{\text{output}}C_{\text{output}}. \quad (6-2)$$

其中 p_{cached} 、 p_{uncached} 和 p_{output} 分别表示三类 token 的单位折算权重。本文不报告绝对货币金额，而是参照 DeepSeek V4 Flash 的缓存输入、非缓存输入和输出 token 计费关系，将三类权重归一化为

$$p_{\text{cached}} : p_{\text{uncached}} : p_{\text{output}} = 0.02 : 1 : 2.$$

因此，表中的价格折算成本单位为 token-equivalent cost/Q，即每题相对 token 成本；数值只用于同一价格假设和同一缓存实现下的方法比较，不能直接解释为人民币或美元金额。主结果表报告原始 token 与轮次指标；成本测算表报告按上述权重得到的价格折算成本。

日志以表6-1所列轮次、缓存输入、非缓存输入、输出和总 token 作为跨方法比较字段。缓存统计优先采用模型服务返回的 cache hit/cache miss 字段；服务端未返回显式缓存字段时，系统使用连续请求最长公共前缀进行估计，并在日志中记录 cache_accounting_source.input_tokens、pre_input_tokens 和 runtime_input_tokens 仅兼容历史日志，不进入主结果表。

6.3 对比方法

主实验比较五个方法。除 Bash Agent 和 Pontis 外，另外三类基线分别代表当前 Text-to-SQL 中常见的多阶段流水线和搜索式推理路线：CHESS 使用检索、schema

selection、候选生成和单元测试组织多 agent 协作^[17]；Alpha-SQL 使用 MCTS 在零样本条件下搜索 SQL 构造动作^[41]；DeepEye-SQL 则将 Text-to-SQL 类比为软件开发过程，使用 value retrieval、schema linking、生成、修正和选择等阶段提高可靠性^[37]。需要说明的是，本文本地实验中的 CHESS 采用快速复现配置，下文记为 CHESS-fast，其结果用于同环境下的系统成本与行为参考，不代表完整四阶段 CHESS 配置的上限性能。

表 6-2 主实验对比方法

方法	目录	定位
Bash Agent	Bash-Agent	无语义层、无专用预处理的通用工具 agent
Pontis	Pontis	语义层与查询智能体
CHESS-fast	CHESS	本文采用的快速复现配置，启用检索与候选生成，不代表完整四阶段配置
Alpha-SQL	Alpha-SQL	基于 MCTS 的 test-time search 方法
DeepEye-SQL	DeepEye-SQL	value retrieval、schema linking、generation、revision、selection pipeline

所有方法使用统一的大模型配置，并在需要 embedding 的环节使用本地 embedding 模型。实验采用零样本设置：各方法在 BIRD dev 评测过程中不使用 BIRD train 集中的 gold SQL、题目答案或 few-shot 示例。所有结果均在同一 BIRD dev 划分、同一执行准确率评测脚本和同一模型配置下统计。

Pontis 在主实验中的在线求解阶段仅开放 find、meta、query 三个工具，prompt 段包括基础身份、工具说明、语义层本体、SQL 规则、护栏规则、数据库上下文、库级 README 和 effort 约束，护栏包括轮次限制、探索检查、SQL 实体检查、桥接表检查、消歧检查、值命中检查和最终 SQL 自检。该配置主要检验语义层检索、元数据读取和 SQL 验证能力，并控制任意文件读取或 shell 探索带来的额外影响。

CHESS 将整体流程组织为四类 agent：Information Retriever 负责关键词抽取、实体检索和上下文检索；Schema Selector 负责表列裁剪；Candidate Generator 负责候选 SQL 生成与修正；Unit Tester 负责生成自然语言单元测试并筛选候选 SQL。CHESS 与 MAC-SQL、CHASE-SQL 等工作类似，核心思想是通过阶段分工、多路径生成或候选验证降低单次生成的不稳定性^[25,40]。本实验采用 CHESS_IR_CG_ONLY_FAST 快速配置。实际运行中启用的是 Information Retriever 与 Candidate Generator：前者使用关键词抽取、entity retrieval 和 top_k=2 的上下文检索，后者只生成单个候选 SQL 并进行一次修正，Schema Selector 和 Unit Tester 未进入本轮实验。因此，本文将该结果记为 CHESS-fast；完整四阶段 CHESS 仍需要在相应配置下单独评估。

6.4 主实验结果

表6-3 给出了五种方法在 BIRD dev 全量评测上的准确率与在线资源消耗。Pontis 在 1534 道题中答对 1003 道，执行准确率为 65.38%，比 Bash Agent 高 5.34 个百分点，与 DeepEye-SQL 的 65.25% 基本持平。该结果说明，Pontis 在引入语义层预处理和较长稳定上下文后，仍保持了与专门 Text-to-SQL pipeline 相近的静态求解能力。由于 Pontis 与 DeepEye-SQL 仅相差 2 道题，本文不将这一差异解释为稳定的准确率优势，而是将

执行准确率作为后续资源结构比较的基本可用性前提。

表 6-3 BIRD dev 主实验结果与在线资源消耗

方法	正确数	准确率	轮次/Q	Cached/Q	Uncached/Q	Output/Q
Pontis	1003	65.38%	11.446	156569.7	6651.9	3174.7
DeepEye-SQL	1001	65.25%	5.339	4253.7	12637.4	5430.2
Alpha-SQL	972	63.36%	12.040	33972.8	18444.5	14181.0
Bash Agent	921	60.04%	6.746	6079.7	1544.8	1897.9
CHESS-fast	900	58.67%	10.795	11191.5	15880.8	5142.9

Bash Agent 在本文中作为轻量工具基线。它没有专门预处理和长期语义层，在线成本较低；但在需要复杂 schema/value grounding 和连接验证的问题上，其执行准确率低于 Pontis 和 DeepEye-SQL。该对比说明，简单工具智能体和语义层式系统代表了不同的成本与能力取舍。

从资源结构看，Pontis 的 cached input 显著高于其他方法，主要来自稳定系统提示、工具说明、库级 README 和数据库上下文。这些稳定前缀能够在同一运行批次中被 token cache 命中。Pontis 的 uncached input 为 6651.9 tokens/Q，低于 DeepEye-SQL、Alpha-SQL 和 CHESS-fast。在支持 prompt cache 的模型服务中，大量稳定前缀可以以较低成本复用，其实际成本结构区别于全部按非缓存输入计费的长上下文方法。长上下文 NL2SQL 研究已经显示，更多上下文可能提升复杂 schema 理解，但同时带来延迟和成本权衡^[34]；cached/uncached 拆分将这类上下文成本显式化。Bash Agent 的成本最低，但准确率也明显低于 Pontis 和 DeepEye-SQL。Alpha-SQL 的输出 token 较高，反映了其多轮搜索和候选推理带来的生成成本。

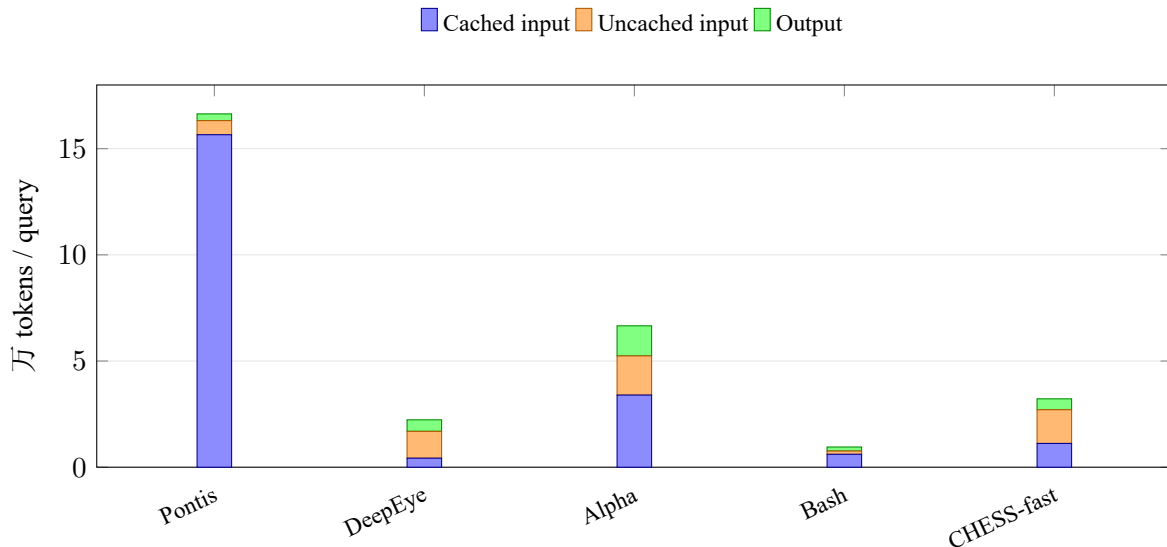


图 6-1 五种方法在线阶段 token 结构对比

表6-4 给出了在线阶段和加入预处理摊销后的价格折算成本。测算使用 0.02 : 1 : 2 的归一化权重，将缓存输入、非缓存输入和输出 token 折算到同一 token-equivalent 成本尺度；embedding 由本地模型完成，不计入 API 成本。Pontis 的预处理摊销后价格折算成本为 18709.1/Q。离线提取增加了预处理成本，但 Pontis 的大部分在线上下文可

缓存，因此摊销后成本仍低于 DeepEye-SQL、Alpha-SQL 和 CHESS-fast。该结果说明，语义层并不一定降低总 token 数，而是改变成本的组成：更多信息被转化为可缓存、可复用的稳定上下文，在线阶段的非缓存探索相应减少。

表 6-4 五种方法的示例价格折算成本对比

方法	在线价格折算成本/Q	含预处理摊销/Q
Bash Agent	5462.1	5462.1
Pontis	16132.8	18709.1
DeepEye-SQL	23582.9	23582.9
CHESS-fast	26390.4	26390.4
Alpha-SQL	47485.9	48254.7

6.5 预处理资源消耗

不同方法的预处理逻辑差异较大。Bash Agent 没有专门预处理；CHESS-fast 主要构建轻量 embedding 索引；DeepEye-SQL 在预处理阶段主要生成 value-level 向量数据库；Alpha-SQL 除少量 embedding 外，还通过 LLM 生成部分数据库预处理资产；Pontis 则在语义层中写入统计、摘要、库级 README、关系线索和向量索引。

表 6-5 预处理阶段平均每库资源消耗

方法	LLM 轮次/库	Cached/库	Uncached/库	Output/库	Embedding/库
Pontis	219.0	1891828.4	120304.7	100566.7	15467.7
DeepEye-SQL	0.0	0.0	0.0	0.0	1240116.0
Alpha-SQL	139.5	87691.6	4994.6	50230.5	3732.6
Bash Agent	0.0	0.0	0.0	0.0	0.0
CHESS-fast	0.0	0.0	0.0	0.0	1059.5

Pontis 的预处理开销最高，主要原因是它把数据库理解过程前移到语义层构建阶段，减少每道题运行时重新探索的需求。实验同时统计预处理总成本和按 BIRD dev 查询数摊销后的平均成本。忽略预处理成本会低估语义层的总体投入；把预处理成本全部计入单题则会低估长期重复查询场景中的摊销价值。Pontis 更适合一个数据库被长期、重复查询的场景；如果只对单个数据库提出极少量问题，预处理摊销不足，成本优势会减弱。

6.6 与基线方法的差异

从系统形态看，CHESS-fast、DeepEye-SQL 和 Alpha-SQL 更接近面向单次基准推理的多阶段方法。它们通常将一次问题处理拆分为 schema linking、value retrieval、候选生成、执行验证、搜索或选择等阶段，目标是在当前 query 上提高 SQL 正确率。CHASE-SQL、MCS-SQL 和 Agentar-Scale-SQL 也体现了同一趋势：通过增加候选路径、选择器或 test-time compute 来换取更稳定的 SQL 生成^[39-40,42]。Pontis 则把重点放在

语义层上：预处理阶段将结构、统计、摘要、关系和消歧信息写入图存储，在线阶段的 agent 通过工具读取这些图实体并执行验证。

这种差异导致两类系统的成本结构不同。基线方法的优势是单题输入较轻，不需要维护长期语义层；但如果同一数据库被持续查询，许多 schema/value 探索会在不同问题之间重复发生。Pontis 需要前期预处理和较长的稳定上下文，换来同一数据库下后续问题对统计、摘要、关系和消歧信息的复用，并支持检查、更新和归因。

同时，多阶段方法的复杂性也可能带来额外误差传播。任务分解的收益取决于各阶段输出质量；如果中间阶段输出错误 schema、错误候选或过窄上下文，后续生成器可能只能在错误前提下优化。因此，复杂 pipeline 的收益需要结合具体模型能力、工具稳定性、候选规模和资源预算共同分析。

从系统目标看，Pontis 与这些基线方法的差异不仅体现在最终准确率上。基线方法更强调在单次 query 上完成尽可能充分的检索、搜索和选择；Pontis 更强调把可复用的数据库理解组织为长期语义层，使后续查询可以复用并审计这些信息。因此，评价重点包括准确率、上下文组织、资源统计和增量维护结构。

6.7 数据库结构与内容变动下的增量更新对比

增量更新指数据库结构或内容发生变化后的系统维护问题。真实数据环境中，表可能新增或删除，列名和类型可能变化，外键约束可能补充，枚举值和数据分布可能漂移，说明文档和业务口径也可能更新。此时，Text-to-SQL 系统需要判断哪些上下文资产仍然有效，哪些索引、摘要、关系和经验需要刷新。不同系统对这些变动的响应取决于其是否维护持久化预处理资产，以及这些资产能否定位到稳定实体和依赖关系。

Bash Agent 没有持久化预处理资产，因此数据库变化后不需要维护离线索引或摘要；但这种低维护成本来自于不复用数据库理解经验，后续每道题仍需要在运行时重新探索 schema、值域和连接路径。对于持续查询的同一数据库，这种模式把更新成本转化为每次在线查询的重复成本。

CHESS-fast、DeepEye-SQL 和 Alpha-SQL 都在一定程度上依赖预处理或检索资产。CHESS-fast 主要维护 schema/value 相关的检索索引；DeepEye-SQL 构建 value-level 向量数据库；Alpha-SQL 除 embedding 外还生成部分数据库预处理摘要。数据库结构或内容变化后，这些资产可以通过重新构建索引、value vector DB、LSH 结构或摘要文件来适配新状态。其优势是实现路径清晰，适合离线 benchmark 复现；局限是更新粒度通常与整库、表或预处理文件绑定，难以直接表达“某个列摘要过期但其他关系仍可用”的细粒度状态。

Pontis 的差异在于更新对象是语义层中的稳定实体和关系。由于统计、摘要、关系、库级 README、消歧经验和 embedding 都挂接在具体图节点或子图上，数据库变动后可以按受影响的表、列和关系子图刷新，而不必默认重建全部上下文。与 Bash Agent 相比，Pontis 把在线重复探索前移为可维护的语义层资产；与 CHESS-fast、DeepEye-SQL

和 Alpha-SQL 相比，Pontis 更强调派生信息的身份、来源和依赖关系，因此更适合同一数据库长期运行、持续查询和逐步修正的场景。

7 评测基准局限性分析

Spider、BIRD、Spider 2.0 等 Text-to-SQL 数据集推动了该领域从教学式 SQL 生成走向更接近真实数据库问答的评测。相比早期只关注跨库 schema 泛化的数据集，BIRD 引入真实数据库内容、外部 evidence、执行结果比对和效率因素；Spider 2.0 进一步把任务扩展到更复杂的数据应用和工作流。这些设计显著提高了基准的真实性，也为不同模型和系统提供了统一、可复现的比较条件。

第六章仍报告 BIRD dev 执行准确率，是因为 EX 是当前 Text-to-SQL 领域最可复现的横向比较指标，能够判断系统是否具备基本求解能力。然而，本文并不将 EX 最大化作为唯一目标。随着 Text-to-SQL 系统从单次 SQL 生成转向工具调用、多阶段搜索和数据智能体，运行成本、上下文复用、错误可解释性和长期维护能力同样会影响系统可用性。

更接近真实场景并不意味着已经完整覆盖真实场景。现有 Text-to-SQL 基准大多仍采用逐题独立的静态单轮设置：给定数据库、自然语言问题和可选 evidence，系统生成一条 SQL，再用 gold SQL 或执行结果进行判定。该协议适合排行榜构建和横向比较，却排除了真实数据分析中常见的意图澄清、业务口径协商、输出格式偏好、历史经验复用和数据库持续变化等因素。因此，执行准确率虽然是必要指标，但其主要衡量系统与数据集标注答案的一致性，并不总是等价于系统是否满足了用户真实分析目标。

这种差距在工具调用智能体出现后变得更加明显。早期 Text-to-SQL 系统的主要错误往往来自 SQL 语法、表列选择和连接路径；而当模型具备较强生成能力并能够读取 schema、执行 SQL、检查样例值之后，失败来源会进一步暴露为问题语义欠定、gold SQL 标注偏好、输出形状差异和数据集隐含先验。因此，静态基准分数的解释需要同时考虑执行准确率的计算流程、SQL 多重可达性、query 语义欠定性、错误归因和标注质量；交互式业务目标与 query log 评测则可以作为静态单轮评测之外的补充协议。

7.1 基本评测流程与执行准确率

现有 Text-to-SQL 基准通常采用单轮输入输出形式。给定数据库 $\mathcal{D}$ 和自然语言问题 q ，系统需要生成一条 SQL 预测结果 $\hat{s}$ ：

$$\hat{s} = f(\mathcal{D}, q). \quad (7-1)$$

在带标注的数据集中，每个样本通常写作 $(\mathcal{D}_i, q_i, s_i^*)$ ，其中 s_i^* 表示人工标注的 gold SQL。模型或系统生成预测 SQL $\hat{s}_i$ 后，评测脚本在同一数据库上分别执行 $\hat{s}_i$ 和 s_i^* ，并比较二者执行结果是否一致。若记 SQL s 在数据库 $\mathcal{D}$ 上的执行结果为 $\text{Exec}(\mathcal{D}, s)$ ，则第 i 道题的执行准确率可写为：

$$EX_i = \mathbb{I}[\text{Exec}(\mathcal{D}_i, \hat{s}_i) = \text{Exec}(\mathcal{D}_i, s_i^*)]. \quad (7-2)$$

整个数据集上的执行准确率为逐题平均：

$$EX = \frac{1}{N} \sum_{i=1}^N EX_i. \quad (7-3)$$

相比直接比较 SQL 字符串或语法树，执行结果比对能够容纳部分语法不同但结果相同的 SQL 写法，因此成为 BIRD、Spider 等基准中常用的核心指标。该评测方式的优点是清晰、可复现、便于不同方法横向比较，也适合构建排行榜和进行大规模自动化评测。

这一评测方案同时隐含若干前提：gold SQL 所对应的执行结果可以代表问题的目标答案，自然语言问题足以确定该目标答案，执行结果一致可以近似表示语义正确，逐题独立平均可以反映系统能力。这些前提限定了静态执行准确率在复杂数据库问答中的解释范围。

执行准确率仍然是跨方法比较中最稳定、最可复现的指标，但其含义更接近“与基准标注的一致性”。BIRD 相比早期基准更接近真实数据库场景，但它仍属于静态基准。它能够衡量模型与 gold SQL 的一致程度，却难以完整衡量模型是否理解并满足用户业务目标。因此，准确率需要结合错误归因、案例分析和资源消耗统计共同解释。

7.2 SQL 多重可达性

在 BIRD 数据集中，同一业务目标在 SQL 层面可能具有多重可达性。为了回答同一个业务问题，可能存在多种执行结果相同、语义相近，甚至在实际应用中更有用的 SQL 写法。BIRD 数据集选择其中一种作为 gold SQL，模型如果生成了另一种业务上合理的 SQL，仍可能因为输出列、排序、格式、数值类型或聚合粒度与 gold SQL 不一致而被判错。

例如，在 BIRD dev 的 card_games/q390 中，问题询问 ID 1-20 的卡牌颜色和格式。Bash Agent 返回了 id, colors, format，其中 id 有助于识别每张卡牌；但 gold SQL 只返回 colors, format，因此额外输出 id 改变了执行结果形状并导致评测错误。而在 superhero/q763 中，问题要求 Abomination 的 attribute value，Bash Agent 返回了 attribute_name, attribute_value，对阅读者更完整，但 gold SQL 只接受 attribute_value。这类案例说明，实际分析中可能需要辅助列，例如 id、名称、属性名或解释性字段；基准则通常只接受 question/evidence 明确要求的最小输出列。

在 toxicology/q226 中，问题要求五位小数百分比，Bash Agent 使用 printf('%.5f', ...) 输出固定五位小数字符串，而 gold SQL 使用 ROUND(..., 5) 输出数值。两者在

自然语言层面都可以解释为“保留五位小数”，但执行结果比对会把字符串和数值视为不同结果。该例体现的是数值格式差异：实际展示可能关注固定小数位格式，基准则可能更偏好数值类型或某种 SQLite 函数写法。

上述例子说明，SQL 多重可达性不仅体现在连接顺序、子查询写法或等价谓词上，也体现在输出列、数值类型和展示格式上。同一个问题可能存在多种业务上合理的 SQL 结果形态；但在静态基准中，只有与 gold SQL 执行结果完全一致的形态会被计为正确。由此可见，执行准确率对结果形态高度敏感，可能将部分业务上可接受但输出形态不同的 SQL 判为错误。

具体而言，执行结果比对会受到输出列、排序、数值格式和类型、聚合粒度、DISTINCT 或额外过滤条件等因素影响。只要这些结果形态与 gold SQL 不一致，执行结果就可能不同；即使预测 SQL 在真实业务场景中合理甚至更有用，也仍会在静态基准中被计为错误。

7.3 Query 语义欠定性

在 BIRD 数据集中，第二类问题来自 query 本身的模糊性。自然语言问题通常是用户基于某种数据库先验写出的简短表达，但这些先验很少完整出现在 query 中。当数据库标注不够丰富、evidence 不够明确，或者同一术语可以映射到多个表列时，agent 需要根据问题文本和数据库内容选择一个最可能的解释。该解释在业务上可能合理，却未必与 BIRD gold SQL 的标注选择一致。

例如，在 BIRD dev 的 codebase_community/q689 中，问题要求找出“last to edit the post with ID 183”的用户。该描述既可能对应 postHistory 中最后一条编辑记录，也可能对应 posts 表中的 last-editor 字段；gold SQL 实际连接的是 owner 字段，这与“last to edit”的自然语言表达存在偏差。不同方法在该题上选择了不同路径，说明问题文本、数据库字段和标注 SQL 之间存在解释空间。

而在 student_club/q1376 中，问题询问 closed events 中 spend-to-budget ratio 最高的 event。一种自然解释是按 event 聚合预算行，计算 $\text{SUM}(\text{spent})/\text{SUM}(\text{amount})$ ；但 gold SQL 按单条 budget row 的 spent/amount 排序，没有进行 event-level aggregation。两种写法对应不同业务粒度：前者回答“某场活动整体预算使用率”，后者回答“某条预算记录比例”。如果用户没有明确说明粒度，系统难以仅凭 query 唯一确定哪一种正确口径。

在 thrombosis_prediction/q1187 中，题目表述为 examined between 两个日期，同时条件来自 GPT 和 ALB。模型可能把 examined date 映射到 Examination.Examination Date 并连接 Laboratory，但 gold SQL 直接使用 Laboratory.Date。该例体现了日期字段来源歧义：两种写法都可以与 examined 语义建立联系，但它们过滤的是不同表中的日期字段，最终行集可能不同。自然语言中的 examined date 存在多个候选表列，需要结合 BIRD 标注口径才能确定。

这些案例说明，部分错误源于 query、schema 和 gold SQL 之间的欠定性。在真实系统中，系统可以通过澄清问题降低该类不确定性，例如确认“这里按 event 聚合还是按预算记录排序”或“examined date 指检查表日期还是实验室检测日期”。静态基准通常不允许这种交互，因此会把语义欠定问题压缩成单次预测错误。

执行结果比对尽管可以缓解 SQL 表面形式差异造成的假阴性问题，但不能消除自然语言 query 的语义欠定性。单句问题不足以唯一决定表选择、字段来源、过滤值和聚合口径时，执行结果比对主要衡量“是否复现了标注者选择的那一种解释”，而对“是否正确理解并满足了用户真实意图”的覆盖有限。

7.4 Bash Agent 错误归因

前两节表明，目前 BIRD 数据集相当一部分错误与评测协议本身有关：一类来自预测 SQL 的输出列、排序、格式或聚合粒度与 gold SQL 不一致，另一类来自自然语言 query 未显式给出足以唯一确定标注口径的业务约束。基于这一判断，本节对 Bash Agent 在 BIRD dev 全量评测中的 613 道错误样本进行归因。Bash Agent 具备 schema 查看、SQL 执行和值检查等基础工具能力，但不依赖专门的 BIRD 先验或多候选选择机制，适合作为通用工具 agent 的错误来源参照。归因结果显示，输出要求和隐含先验相关错误占多数，进一步说明静态执行准确率会把标注输出形态、数据集隐含先验和真实数据库探索错误混合计入同一种“错误”。

表 7-1 Bash Agent 错误样本归因分类

类别	数量	占错误样本比例	含义
BIRD SQL 风格与输出要求	246	40.1%	SQL 可能在业务上合理，但输出列、排序、格式、聚合粒度或 DISTINCT/LIMIT 写法与 gold SQL 不一致。
数据集隐含先验	222	36.2%	仅凭 query 和数据库难以唯一确定 gold SQL 的表列选择、口径或标注偏好，需要额外数据集先验。
数据库探索可修复	145	23.7%	通过更充分的 schema、value、join 和样例验证，有机会修正的数据库理解错误。

表7-1 显示，Bash Agent 错误样本中约四分之一属于相对直接的数据库探索可修复问题；更多样本落在 BIRD 输出要求和数据集隐含先验两类，二者合计占错误样本的 76.3%。其中，BIRD SQL 风格与输出要求主要对应 SQL 多重可达性中的结果形态问题；数据集隐含先验则主要对应 query 语义欠定性和标注口径选择问题。Text-to-SQL 基准中的错误来源较为混合：有些错误可以通过更好的表列定位、值验证和连接路径搜索解决；有些错误则来自题面没有显式给出的标注者偏好、输出形状约束或业务粒度选择。后两类问题对额外工具调用和模型推理的收益提出了限制。

Bash Agent 的错误归因为前述分析提供了定量佐证：当前 Text-to-SQL 评测基准中的错误来源较为混合，既包括可以通过更充分探索和验证缓解的数据库理解问题，也包括 SQL 多重可达性、query 语义欠定性和标注口径偏好。后几类问题难以仅依靠改进模型生成能力完全消除，需要结合更丰富的评测设置、交互式澄清和人工审查共同

分析。

7.5 标注错误与排行榜稳定性

Bash Agent 的错误归因揭示了输出要求、隐含先验和问题欠定性在 BIRD dev 错题中的占比；这类问题主要体现为自然语言 query 的欠定性、SQL 多重可达性以及静态评测对 gold SQL 输出形态的依赖。而 Jin 等人的人工审计研究进一步表明，当前 Text-to-SQL 基准中还存在大量标注错误。这些错误主要来自 gold SQL 与自然语言问题逻辑、数据库 schema、数据语义或相关领域知识之间的不一致^[44]。

研究对 BIRD Mini-Dev 和 Spider 2.0-Snow 的审计显示，二者分别存在 52.8% 和 62.8% 的标注错误率；在 BIRD Dev 的随机 100 题子集上，纠正标注后 16 个开源 Text-to-SQL agent 的相对执行准确率变化范围达到 -7% 至 31%，排名变化范围达到 -9 至 +9^[44]。

具体到 BIRD Dev 随机子集，Jin 等人修正了 48 道样本，其中包括 41 处 SQL 修正、19 处问题修正、17 处外部知识修正、1 处 schema 修正以及 6 处数据库内容修正；修正后，CHESS 在该子集上的执行准确率从 62% 提升到 81%，并从中游排名升至第一，而部分原本排名靠前的方法则出现下降^[44]。这些结果说明，静态执行准确率会同时受到模型能力、系统设计、数据集标注质量和标注偏好的影响。因而，在解释 BIRD 等基准结果时，准确率适合作为可复现的横向比较指标；对真实业务正确性的判断还需要结合错误归因、案例审查和任务语义分析。

7.6 从静态评测到交互式业务目标

现有评测基准通常默认 Text-to-SQL 是一次性输入输出任务：用户给出一个自然语言 query，系统一次性生成 SQL，并用 gold SQL 或其执行结果进行判定。此过程可以用 $\hat{s} = f(\mathcal{D}, q)$ 概括，其中 q 被视为完整且充分的问题描述。但在真实数据分析场景中，用户问题经常存在歧义或信息缺失，系统需要通过追问、澄清和执行验证来补足时间范围、指标口径、输出粒度、实体定义或过滤条件。因此，交互式评测可以作为现有 Text-to-SQL 静态评测的重要补充。

交互式 Text-to-SQL 更接近一个离散时间状态转移过程。令 $h_0 = q$ 表示初始交互状态，即用户最初给出的自然语言问题；令 $\mathcal{K}$ 表示当前可用的语义层或外部上下文，若系统没有维护这类信息，则可将 $\mathcal{K}$ 视为空。第 t 轮系统根据数据库、语义层和当前交互状态生成动作 a_t ，动作可以是澄清问题或工具调用；随后系统根据用户回复、工具返回结果或执行反馈等观测 o_t 更新交互状态：

$$a_t = \pi(\mathcal{D}, \mathcal{K}, h_{t-1}), \quad h_t = \Phi(h_{t-1}, a_t, o_t). \quad (7-4)$$

当 a_t 是澄清问题时， o_t 对应用户补充的约束；当 a_t 是工具调用时， o_t 对应数据库查

询、schema 检索或执行验证结果。经过 τ 轮交互后，系统从最终状态生成 SQL，并将其作为终止动作提交：

$$\hat{s} = g(\mathcal{D}, \mathcal{K}, h_\tau), \quad a_{\tau+1} = \text{Final}(\hat{s}). \quad (7-5)$$

在这一表示下，静态评测可以视为 $\mathcal{K}$ 为空或固定、 $\tau = 0$ 或不允许澄清动作的特例，即系统直接从 q 生成 $\hat{s}$ 。真实使用中的最终 SQL 由数据库、语义层、初始问题以及后续交互共同确定。

例如，当用户询问 **ratio** 时，系统可以确认是否乘以 100、是否保留小数、按行计算还是按实体聚合；当用户询问 **latest** 或 **last edit** 时，系统可以确认使用创建时间、更新时间、历史记录还是某个当前字段；当用户询问学校、球员或活动时，系统可以确认输出 id、名称还是完整解释列。这类澄清在真实业务中较为常见，但在静态基准中通常被禁止。因此，静态基准会把一部分本应通过澄清解决的问题压缩为单轮预测错误。

Pontis 的图存储、工具和护栏能够支持智能体查找相关表列、读取统计和样例值、执行 SQL 验证、记录消歧经验，并为用户澄清和经验维护保留系统接口。BIRD dev 实验验证了该语义层在静态基准下的效果；其长期作用在于支撑交互式目标澄清和持续信息维护。

7.7 独立同分布假设与 Query Log 评测

现有 Text-to-SQL 基准还隐含了另一项评测假设：每个 query 通常被视为相互独立的样本，系统在回答当前问题时不能利用同一测试数据库中其他问题的 gold SQL、用户反馈或历史修正经验。这一设定满足了学习任务中的独立同分布假设，有利于标准化比较，但从真实数据分析系统角度看，逐题独立设置与长期数据库使用场景存在差异。

真实业务中的 Text-to-SQL 系统往往面向一个长期存在的数据库。用户围绕同一数据库持续提出相关问题，前一次查询确认过的字段含义、连接路径、指标口径和输出偏好，理应成为后续查询的上下文资产。若将数据库之外可持续维护的语义层记为 $\mathcal{K}_t$ ，则长期使用场景可以写成数据库与语义层共同参与的状态转移过程：

$$a_t = \pi(\mathcal{D}, \mathcal{K}_{t-1}, q_t), \quad (\hat{s}_t, \mathcal{K}_t) = T(\mathcal{D}, \mathcal{K}_{t-1}, q_t, a_t, r_t). \quad (7-6)$$

其中 $\mathcal{K}_{t-1}$ 表示第 t 次查询前已经沉淀的语义信息，可以包括历史 query、已确认 SQL、用户修正、澄清记录、字段口径说明、连接路径经验和执行验证结果。 a_t 表示系统在本轮采取的动作，既可以是检索表列、执行 SQL、生成最终 SQL，也可以是把用户确认、错误归因或新的字段口径写回语义层。 r_t 表示第 t 次查询后的确认结果，可以来自用户修正、人工确认、执行验证反馈或错误归因记录。当 $\mathcal{K}_{t-1}$ 为空且不允许写回语义层时，上述形式退化为逐题独立的静态评测。

例如，当用户已经确认某个 **ratio** 应按实体聚合、某个 **last edit** 应使用历史记录表、某个枚举值需要映射到特定编码列时，系统在后续相似问题中重复推断相同语义会降低使用效率。因此，在线业务场景中的 **query** 更接近同一数据库下按时间到达的 **query log**，与相互独立同分布样本存在明显差异。

BIRD 采用跨数据库划分训练集和开发集的方式，每个数据库对应一定数量的自然语言问题和 **gold SQL**。这种划分有利于考察系统在未见数据库上的泛化能力，并降低对特定测试数据库的直接记忆。随着大模型基础能力增强，部分 **Text-to-SQL** 场景已经不再依赖针对单一数据集的任务级训练；此时，跨数据库划分更适合评估通用泛化能力，而较难刻画同一业务数据库中经验积累、语义维护和持续修正的价值。因此，除跨数据库泛化评测外，还可以进一步关注模型或智能体框架如何在“同一业务数据库内进行经验积累和持续学习”。以 BIRD 数据集为例，一种可行的补充划分方式是把同一数据库中的问题分为历史集、开发集和测试集，允许系统先通过部分历史问题积累对该数据库的理解和经验，再在后续问题中验证这些经验的复用效果和增量更新能力。

因此，未来评测可以在保持静态 **benchmark** 可复现性的同时，引入补充性的 **query log** 评测协议。**Query log** 协议可以作为传统 **dev/test** 划分之外的系统能力评估：给定同一数据库上的历史问题、已确认 **SQL**、用户修正或澄清记录，系统需要将这些经验转化为可复用信息，并在后续问题中提高准确率、降低工具探索轮次和减少歧义错误。这样的评测更适合衡量数据智能体的长期学习能力、信息维护能力和增量更新能力。

8 总结与展望

本文围绕自然语言数据分析中上下文组织困难、模式与值落地不稳定、工具验证成本较高以及数据库理解经验难以复用等问题，设计并实现了 Pontis 语义层系统。系统以 Neo4j 属性图作为持久化后端，将文件、数据库、表、列、外键、列统计、语义摘要、关系线索和经验记录统一组织为可查询实体；同时通过自动化提取模块、探索智能体和语义向量索引，将原始数据资产转化为可被查询智能体逐步读取和验证的上下文。在线阶段，Pontis 将语义层检索、元数据查看和 SQL 查询封装为工具，并结合实体检查、连接检查、消歧检查和值命中检查等护栏机制，使智能体能够在生成最终 SQL 前显式核实相关证据。

实验部分以 BIRD dev 全量 1534 道题为主要评测对象，在零样本设置下比较了 Pontis 与 Bash Agent、CHESS-fast、Alpha-SQL 和 DeepEye-SQL 等方法。结果显示，Pontis 取得 65.38% 的执行准确率，表明语义层式系统在静态 Text-to-SQL 评测中具备基本求解能力。更重要的是，资源统计揭示了 Pontis 与典型多阶段方法不同的成本结构：Pontis 的在线缓存输入较高，但非缓存输入相对可控；在考虑预处理摊销和缓存输入折算后，其成本结构体现出将数据库理解前移到可复用语义层中的系统取舍。对于同一数据库被反复查询的场景，这种设计能够减少在线阶段重复探索 schema、值分布和关系路径的需求。

本文进一步分析了静态执行准确率在评价数据智能体时的边界。BIRD 等基准为 Text-to-SQL 系统提供了清晰、可复现的横向比较方式，但单轮执行准确率会把数据库探索不足、输出格式偏好、自然语言语义欠定和标注口径差异共同压缩为同一种错误。因而，执行准确率可以作为判断系统基本能力的重要指标，但不足以完整刻画长期数据智能体的实际可用性。面向真实数据分析场景，系统还需要同时关注上下文复用、资源成本、错误归因、可维护性以及数据库变化下的增量更新能力。Pontis 的实现和实验表明，在原始数据与 LLM 智能体之间维护可查询、可复用的语义层，是支撑这类能力的一种可行系统路径。

致 谢

值此论文完成之际，谨向在本课题研究和论文撰写过程中给予帮助的老师、同学和家人表示诚挚感谢。

首先，感谢指导教师在选题、系统设计、实验分析和论文修改过程中给予的指导与建议。老师严谨的治学态度和对问题本质的持续追问，使我在完成系统实现的同时，也更加重视实验结论的边界、工程取舍和论文表述的准确性。

其次，感谢实验室和同学们在课题讨论、代码调试和论文写作过程中提供的帮助。与大家交流使我能够从不同角度审视 Text-to-SQL 系统中的上下文组织、工具调用和评测问题，也帮助我不断改进系统实现和实验分析。

最后，感谢家人在学习和生活上给予的理解与支持。正是这些支持使我能够较为专注地完成毕业设计工作。本文仍有不足之处，相关问题也将成为我今后继续学习和改进的方向。

参考文献

- [1] YU T, ZHANG R, YANG K, et al. Spider: a large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task[C]//Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. 2018: 3911-3921.
- [2] LI J, HUI B, QU G, et al. Can LLM already serve as A database interface? A big bench for large-scale database grounded text-to-SQLs[C/OL]//Advances in Neural Information Processing Systems: Vol. 36. 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/83fc8fab1710363050bbd1d4b8cc0021-Abstract-Datasets_and_Benchmarks.html.
- [3] LEI F, CHEN J, YE Y, et al. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows[C/OL]//Proceedings of the 13th International Conference on Learning Representations. 2025. <https://openreview.net/forum?id=XmProj9cPs>.
- [4] SAHU G, PURI A, RODRIGUEZ J, et al. InsightBench: Evaluating Business Analytics Agents Through Multi-Step Insight Generation[A/OL]. arXiv (2025). <https://arxiv.org/abs/2407.06423>. DOI: 10.48550/arXiv.2407.06423.
- [5] EPIC Data Lab, UC Berkeley, Hasura PromptQL. DAB: Data Agent Benchmark[EB/OL]. Project website, 2026. <https://ucbepic.github.io/DataAgentBench/>.
- [6] Microsoft. Fabric data agent concepts[EB/OL]. Microsoft Learn, 2026. <https://learn.microsoft.com/en-us/fabric/data-science/concept-data-agent>.
- [7] Snowflake. Cortex agents[EB/OL]. Snowflake Documentation, 2026. <https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-agents>.
- [8] Databricks. What is a genie space[EB/OL]. Databricks Documentation, 2026. <https://docs.databricks.com/gcp/en/genie/index.html>.
- [9] Tableau. Agentic analytics: a new paradigm for business intelligence[EB/OL]. Tableau Blog, 2025. <https://www.tableau.com/blog/agentic-analytics-new-paradigm-for-business-intelligence>.
- [10] CHEN A, BUNDELE M, AHLAWAT G, et al. Text-to-SQL for Enterprise Data Analytics[A/OL]. arXiv (2025). <https://arxiv.org/abs/2507.14372>. DOI: 10.48550/arXiv.2507.14372.
- [11] JOHNSON J. QueryGPT - Natural Language to SQL using Generative AI[EB/OL]. Uber Blog, 2024. <https://www.uber.com/en-EG/blog/query-gpt/>.
- [12] MAAMARI K, ABUBAKER F, JAROSLAWICZ D, et al. The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models[J/OL]. OpenReview preprint, 2024. <https://openreview.net/forum?id=fglyh5pa7d#discussion>.
- [13] SHKAPENYUK V, SRIVASTAVA D, JOHNSON T, et al. Automatic Metadata Extraction for Text-to-SQL[A/OL]. arXiv (2025). <https://arxiv.org/abs/2505.19988>. DOI: 10.48550/arXiv.2505.19988.
- [14] WANG X, COSTA M, KOVACEVA J, et al. Plugging Schema Graph into Multi-Table QA: A Human-Guided Framework for Reducing LLM Reliance[A/OL]. arXiv (2025). <https://arxiv.org/abs/2506.04427>. DOI: 10.48550/arXiv.2506.04427.
- [15] RISSAKI A, FOUNTALIS I, VASILOGLOU N, et al. Towards Agentic Schema Refinement[A/OL]. arXiv (2024). <https://arxiv.org/abs/2412.07786>. DOI: 10.48550/arXiv.2412.07786.
- [16] SUN R, ARIK S O, MUZIO A, et al. SQL-PaLM: Improved Large Language Model Adaptation for Text-to-SQL (extended)[A/OL]. arXiv (2024). <https://arxiv.org/abs/2306.00739>. DOI: 10.48550/arXiv.2306.00739.
- [17] TALAEI S, POURREZA M, CHANG Y C, et al. CHESS: Contextual Harnessing for Efficient SQL

- Synthesis[C/OL]//Workshop on Multi-Agent Systems in the Era of Foundation Models: Opportunities, Challenges and Futures, ICML 2025. 2025. <https://icml.cc/virtual/2025/49375>.
- [18] DENG M, RAMACHANDRAN A, XU C, et al. ReFoRCE: A Text-to-SQL Agent with Self-Refinement, Consensus Enforcement, and Column Exploration[A/OL]. arXiv (2025). <https://arxiv.org/abs/2502.00675>. DOI: 10.48550/arXiv.2502.00675.
- [19] CAO B, LIAO W, SUN Y, et al. APEX-SQL: Talking to the data via Agentic Exploration for Text-to-SQL[A/OL]. arXiv (2026). <https://arxiv.org/abs/2602.16720>. DOI: 10.48550/arXiv.2602.16720.
- [20] ZHONG V, XIONG C, SOCHER R. Seq2SQL: generating structured queries from natural language using reinforcement learning[C]//Proceedings of the 2017 ACM SIGMOD international conference on management of data. 2017: 236-247.
- [21] WANG B, SHIN R, LIU X, et al. RAT-SQL: relation-aware schema encoding and linking for text-to-SQL parsers[C]//Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. 2020: 7567-7578.
- [22] GAO Y, LIU H, JIA X, et al. C3: zero-shot text-to-SQL with ChatGPT[A/OL]. arXiv (2024). <https://arxiv.org/abs/2307.07306>.
- [23] GAO D, WANG H, LI Y, et al. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation[J/OL]. Proceedings of the VLDB Endowment, 2024, 17(5): 1132-1145. <https://doi.org/10.14778/3641204.3641221>.
- [24] POURREZA M, RAFIEI D. DIN-SQL: decomposed in-context learning of text-to-SQL with self-correction[C/OL]//Advances in Neural Information Processing Systems: Vol. 36. 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/72223cc66f63ca1aa59edaec1b3670e6-Abstract-Conference.html.
- [25] WANG B, REN C, YANG J, et al. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL[C/OL]//Proceedings of the 31st International Conference on Computational Linguistics. Abu Dhabi, UAE: Association for Computational Linguistics, 2025: 540-557. <https://aclanthology.org/2025.coling-main.36/>.
- [26] CHEN P B, WENZ F, ZHANG Y, et al. BEAVER: An Enterprise Benchmark for Text-to-SQL[A/OL]. arXiv (2025). <https://arxiv.org/abs/2409.02038>. DOI: 10.48550/arXiv.2409.02038.
- [27] ZHOU X, SUN Z, LI G. DB-GPT: Large Language Model Meets Database[J/OL]. Data Science and Engineering, 2024, 9: 102-111. <https://doi.org/10.1007/s41019-023-00235-6>.
- [28] ZHANG W, SHEN Y, TAN Z, et al. Data-Copilot: Bridging Billions of Data and Humans with Autonomous Workflow[A/OL]. arXiv (2023). <https://arxiv.org/abs/2306.07209>. DOI: 10.48550/arXiv.2306.07209.
- [29] DIBIA V. LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics using Large Language Models[C/OL]//Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations). Toronto, Canada: Association for Computational Linguistics, 2023: 113-126. <https://aclanthology.org/2023.acl-demo.11/>. DOI: 10.48550/arXiv.2303.02927.
- [30] HONG S, LIN Y, LIU B, et al. Data Interpreter: An LLM Agent for Data Science[A/OL]. arXiv (2024). <https://arxiv.org/abs/2402.18679>. DOI: 10.48550/arXiv.2402.18679.
- [31] BISWAL A, PATEL L, JHA S, et al. Text2SQL is Not Enough: Unifying AI and Databases with TAG[C/OL]//Proceedings of the 15th Annual Conference on Innovative Data Systems Research. Amsterdam, The Netherlands, 2025. <https://www.vldb.org/cidrdb/papers/2025/p11-biswal.pdf>.
- [32] NAHID M M H, RAFIEI D, ZHANG W, et al. Rethinking Schema Linking: A Context-Aware Bidirectional Retrieval Approach for Text-to-SQL[A/OL]. arXiv (2025). <https://arxiv.org/abs/2510.14296>.

- DOI: 10.48550/arXiv.2510.14296.
- [33] EBEN J, AHMAD A, LAU S. RASL: Retrieval Augmented Schema Linking for Massive Database Text-to-SQL[A/OL]. arXiv (2025). <https://arxiv.org/abs/2507.23104>. DOI: 10.48550/arXiv.2507.23104.
 - [34] CHUNG Y, KAKKAR G T, GAN Y, et al. Is Long Context All You Need? Leveraging LLM’s Extended Context for NL2SQL[J/OL]. Proceedings of the VLDB Endowment, 2025, 18(8): 2735-2747. <https://www.vldb.org/pvldb/vol18/p2735-ozcan.pdf>. DOI: 10.14778/3742728.3742761.
 - [35] LIU N F, LIN K, HEWITT J, et al. Lost in the Middle: How Language Models Use Long Contexts [J/OL]. Transactions of the Association for Computational Linguistics, 2024, 12: 157-173. https://doi.org/10.1162/tacl_a_00638.
 - [36] CHEN Z, LI H, ZHANG X, et al. RubikSQL: Lifelong Learning Agentic Knowledge Base as an Industrial NL2SQL System[A/OL]. arXiv (2025). <https://arxiv.org/abs/2508.17590>. DOI: 10.48550/arXiv.2508.17590.
 - [37] LI B, CHEN C, XUE Z, et al. DeepEye-SQL: A Software-Engineering-Inspired Text-to-SQL Framework[J/OL]. Proceedings of the ACM on Management of Data, 2026, 4(3). <https://doi.org/10.1145/3802035>.
 - [38] WANG X, WEI J, SCHUURMANS D, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models[A/OL]. arXiv (2023). <https://arxiv.org/abs/2203.11171>. DOI: 10.48550/arXiv.2203.11171.
 - [39] LEE D, PARK C, KIM J, et al. MCS-SQL: Leveraging Multiple Prompts and Multiple-Choice Selection For Text-to-SQL Generation[C/OL]//Proceedings of the 31st International Conference on Computational Linguistics. Abu Dhabi, UAE: Association for Computational Linguistics, 2025: 337-353. <https://aclanthology.org/2025.coling-main.24/>.
 - [40] POURREZA M, LI H, SUN R, et al. CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL[C/OL]//Proceedings of the 13th International Conference on Learning Representations. 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/974ff7b5bf08dbf9400b5d599a39c77f-Abstract-Conference.html.
 - [41] LI B, ZHANG J, FAN J, et al. Alpha-SQL: Zero-Shot Text-to-SQL using Monte Carlo Tree Search [C/OL]//Proceedings of Machine Learning Research: Vol. 267 Proceedings of the 42nd International Conference on Machine Learning. Vancouver, Canada: PMLR, 2025: 36810-36830. <https://proceedings.mlr.press/v267/li25dt.html>.
 - [42] WANG P, SUN B, DONG X, et al. Agentar-Scale-SQL: Advancing Text-to-SQL through Orchestrated Test-Time Scaling[A/OL]. arXiv (2025). <https://arxiv.org/abs/2509.24403>. DOI: 10.48550/arXiv.2509.24403.
 - [43] WRETLAD N, RISEBY F, BISWAS R, et al. Understanding the Effects of Noise in Text-to-SQL: An Examination of the BIRD-Bench Benchmark[C/OL]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers). Bangkok, Thailand: Association for Computational Linguistics, 2024: 356-369. <https://aclanthology.org/2024.acl-short.34/>. DOI: 10.18653/v1/2024.acl-short.34.
 - [44] JIN T, CHOI Y, ZHU Y, et al. Pervasive Annotation Errors Break Text-to-SQL Benchmarks and Leaderboards[A/OL]. arXiv (2026). <https://arxiv.org/abs/2601.08778>. DOI: 10.48550/arXiv.2601.08778.
 - [45] ABEDJAN Z, GOLAB L, NAUMANN F. Profiling relational data: a survey[J/OL]. The VLDB Journal, 2015, 24(4): 557-581. <https://doi.org/10.1007/s00778-015-0389-y>.
 - [46] FERNANDEZ R C, ABEDJAN Z, KOKO F, et al. Aurum: a data discovery system[C/OL]//2018 IEEE

- 34th International Conference on Data Engineering (ICDE). Paris, France: IEEE, 2018: 1001-1012. <https://doi.org/10.1109/ICDE.2018.00094>.
- [47] SCHICK T, DWIVEDI-YU J, DESSI R, et al. Toolformer: language models can teach themselves to use tools[J]. Advances in Neural Information Processing Systems, 2023, 36: 68639-68651.
- [48] YAO S, ZHAO J, YU D, et al. ReAct: synergizing reasoning and acting in language models[C]// International Conference on Learning Representations. 2023.
- [49] FAN G, WANG J, LI Y, et al. Semantics-aware dataset discovery from data lakes with contextualized column-based representation learning[J/OL]. Proceedings of the VLDB Endowment, 2023, 16(7): 1726-1739. <https://doi.org/10.14778/3587136.3587146>.
- [50] HULSEBOS M, HU K, BAKKER M, et al. Sherlock: a deep learning approach to semantic data type detection[C/OL]//Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. Anchorage, AK, USA: ACM, 2019: 1500-1508. <https://doi.org/10.1145/3292500.3330993>.

附录 A Pontis 代码模块说明

本文所设计与实现的 Pontis 系统代码已开源，仓库地址为：<https://github.com/rhico re/Pontis>。

Pontis 的主要代码结构如下：

```
Pontis/
|-- storage/          # Workspace、Neo4j 图接口和发布触发机制
|-- extractor/        # 自动提取模块和流水线执行器
|-- explorer/         # agent 预处理模块
|-- tool/             # agent-facing 工具实现
|-- agent/            # LLM 循环、prompt、工具注册、guardrails
|-- utils/            # token、embedding 和通用运行统计辅助函数
|-- scripts/BIRD/     # BIRD 抽取和 benchmark 脚本
|-- docs/             # 架构、性能和实验分析文档
|-- pontis_cli.py      # 命令行入口
`-- pontis.yml         # 工作空间与 Neo4j 图配置
```

Pontis 的主要工具和模块可通过以下命令查看：

```
python -m extractor list
pontis <project_path>
python -m scripts.BIRD.extract --help
python -m scripts.BIRD.run_bird_benchmark --db card_games --limit 5
```